Efficiently Linking Unstructured Data for Multistep Reasoning

Anonymous Author(s)

Abstract

Modern data systems leverage agentic techniques to reason over and integrate unstructured data. Typical agentic data pipelines perform data extraction, integration, and ranking, then feed promising answers to complex reasoning stages, e.g., identifying candidate leads for scientific discovery. The **core retrieval problem** in these workflows jointly executes multi-attribute filtering, multi-vector search, relational joins, and embedding-similarity joins. Given a query plan and monotone scoring function, our DASE query engine holistically constructs and ranks candidate evidence tuples. It comprises (i) a multistep reasoning query model over multiple structured predicates, vectors, and relational links; (ii) SemJI, a sparse materialized embedding-similarity join index for near-neighbor pairs; and (iii) an execution layer that combines predicate-aware ANN traversal, batched access, and threshold-based scoring.

We experimentally demonstrate that DASE provides significant speedups over a variety of workloads. On scientific-discovery workloads, DASE retrieves candidate evidence for multistep reasoning queries 6–46× faster than RDBMS, reranking, and vector-database baselines at comparable recall. DASE can also be used to prefilter queries that require semantic-operator post-processing with LLMs, with a cost savings of up to 4×.

ACM Reference Format:
Anonymous Author(s). 2027. Efficiently Linking Unstructured Data for Multistep Reasoning. In *Proceedings of SIGMOD Conference on Management of Data (SIGMOD '27).* ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Data systems have begun to incorporate agentic and LLM technologies to extract, integrate, and reason about data from multiple unstructured sources. Example *agentic data engineering systems* include "deep research" [20, 62, 66] and "wide search" agents [8, 21, 25, 26, 56, 60], systems enabling scientific discovery and literature synthesis [3, 17, 19, 30], LLM-based extract-transform-load tools for unstructured data [1, 45], and unstructured-to-structured query systems based on *semantic operators* [31, 38, 43, 47]. Although these systems have fundamental architectural differences, they generally share a core ranked retrieval operation: given a user query, they decompose it into plans, using *multistep reasoning* retrieval to find, rank, and assemble entity combinations that satisfy complex search criteria. In a second phase, those integrated results are fed into agentic tools or LLM prompts. Agent-generated retrieval and integration workloads require novel query execution techniques, in line with the call for *agent-first* architectures [32]. This paper targets the initial data-retrieval task and layer: given a planned query and scoring function, we construct and rank candidate evidence tuples. This paper presents the query execution component of our system; our agentic planner (which incorporates chain-of-thought [54] and ReAct [62] to break prompts into substeps suitable for agentic processing) is discussed in a concurrently submitted work.

Linking unstructured knowledge. Our team, in collaboration with chemists and life scientists, is building an agentic data engineering tool to accelerate scientific discovery. Two initial applications involve discovering candidate molecules with pharmaceutical benefits and engineering RNA sequences with therapeutic properties. Both tasks require reasoning over published papers from the scientific literature. This first involves data preprocessing, including (1) indexing document paragraphs with embeddings, as in retrieval-augmented generation (RAG) [16], and (2) extracting knowledge-graph-like key-value annotations of those paragraphs [40, 44] and indexing the extracted values using domain-dependent encodings or embeddings (e.g., text or image embeddings; chemical embeddings [10]; molecular structure encodings [42, 55]). Accompanying textual passages, as well as knowledge graph properties such as outcomes or experimental settings, might also be embedded using standard text or multimodal embeddings.

Example 1.1 (Agentic data engineering for molecular discovery). The process of discovering and optimizing molecules or RNA starts with an information gathering stage, i.e., a query that assembles candidate results. Since evidence is often fragmented across documents and their extracted annotations, a query about a molecule might (1) approximately match against a ChemBERTa embedding, extracted from the text into a knowledge graph, but linked back to the paragraph in which it appears; (2) match on an extracted `organism`; (3) match against embeddings of experimental conclusions. All of these matches must be joined to ensure they are within a common paper and ranked using a function that **combines all of the individual matches**. Finally, a ranked list of promising candidate molecules from this query stage is often fed into additional tools, e.g., to optimize their chemistry, or to generate new molecules using fine-tuned LLMs.

Such queries may also need to link information across documents based on approximate matches in their content:

Example 1.2. Building upon Example 1.1, a scientific-discovery query might be based on the observation that small molecules may have similar characteristics even if they are not particularly similar in ChemBERTa embedding space. Thus we might seek to *find related molecules*, by matching on a first document, then **linking and joining via text description embedding** with

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD '27, Huntington Beach, CA USA
© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

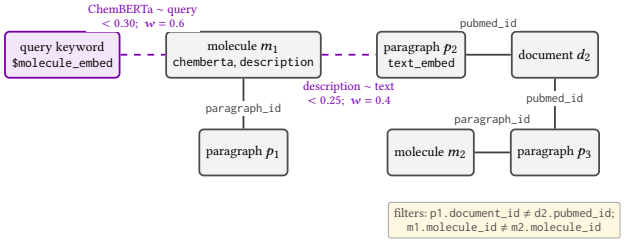


Figure 1: A graph pattern for the query described in Example 1.2, corresponding to agentic planner-generated SQL (Listing 1). The leftmost keyword-input leaf, `$molecule_embed`, matches molecule variable m_1; dashed purple edges are thresholded embedding-similarity predicates whose distances form the ranking score, and solid edges are exact joins.

other documents, then traversing (joining) to their extracted molecular descriptions. Figure 1 visualizes this. We will formalize and revisit this query in subsequent examples.

The retrieval task is not just to locate relevant records step-by-step. A candidate must be evaluated jointly against the query's criteria, even when the evidence for those criteria is scattered across sources. This requires linking the relevant records and ranking the resulting combinations holistically. As collections grow, the system must search a large space of candidate–evidence combinations, while returning only a shortlist for downstream reasoning.

Concretely, a query can combine structured predicates, exact joins, and thresholded embedding-similarity joins. It returns the top-k evidence tuples according to a scoring function that combines multiple similarity signals. Conventional vector DBMSs such as Pinecone or Weaviate [41, 53] do not natively plan multi-table joins and joint multi-signal ranking, so an external query processor must stitch such operations together. Milvus [50], designed for hybrid search, supports logical predicates and vector retrieval but does not provide relational join planning. More recent systems, including Nova [7] and Presto Vector Search [59], integrate vector retrieval with relational joins, but do not provide dedicated execution support for joint top-k ranking of cross-table evidence tuples under a score combining query-to-row and row-to-row embedding similarities. We therefore study how to implement multistep reasoning queries in a relational DBMS with approximate-nearest-neighbor (ANN) vector-index support.

As we demonstrate in Section 5, standard RDBMS execution strategies for combining predicates with approximate-nearest-neighbor search [18, 33] perform poorly on join-heavy multistep reasoning queries and other semantic-query tasks [27]. Conversely, information-retrieval-style reranking [6, 37] exhibits low recall. Multistep reasoning queries introduce challenges in query indexing, optimization, and execution, which we address with the following contributions:

- **Query building block for agentic data engineering.** We formalize *ranked multistep reasoning queries* that support structured filters, vector scores, and cross-table joins.
- **Indexing strategy.** We develop SemJI, a materialized embedding-similarity join index that represents near-neighbor pairs as a relational access path for top-k multistep reasoning queries.
- **Algorithm/system co-design.** We extend PostgreSQL's ANN-index support with batched access and develop DASE, which combines predicate-aware ANN traversal, filtered score streams, and threshold-based aggregation to jointly optimize attribute predicates and vector scoring across relational joins.
- **Evaluation.** We evaluate throughput and recall on semantic-query benchmarks and a scientific-discovery workload, including an optional transformed-space SemJI construction for anisotropic or shared-context endpoint embeddings that make a raw distribution-derived materialization radius unreliable.

To ensure reproducibility, our code, data, and full technical report are available at https://anonymous.4open.science/r/DASE-2C61/.

2 Query Model and Index Strategy

This paper targets query answering tasks over unstructured data that has been indexed and annotated within a local relational DBMS, which may include both approximate nearest-neighbor search over embeddings and logical predicates over relationships.

Data model. Our underlying data model is based on object-relational tables, representing content extracted from documents and files. Tables are linked through foreign key relationships. Using a combination of user-defined functions and semantic operators [38, 45], indexed data may be parsed, mapped to an embedding, and then analyzed using LLM operators such as *semantic map* to extract structured information from the parsed content; *semantic filter* to test tuples' content against a prompt, etc. The results of semantic maps may be saved as labeled *annotations* in a structured relational table, in a controlled vocabulary, with explicit provenance links to the source content, and with embeddings over their values.

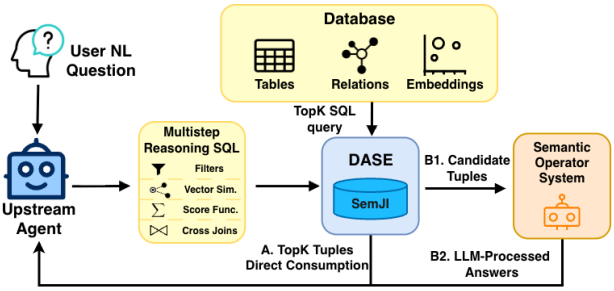


Figure 2: DASE interactions with other system components.

2.1 Top-k Multistep Reasoning Queries

Our target use case involves an upper-layer agentic system, performing deep or wide search, or another form of multistep reasoning over a user's natural-language question. This creates an agentic pipeline, in which the agent, or a planning frontend, creates an initial retrieval and integration task involving a structured query over a local dataset. This *multistep reasoning query* combines multi-attribute filtering, multi-vector similarity scoring, and cross-table joins, together with a scoring function that aggregates these signals into a unified ranking objective. Our DASE engine executes this query and returns the top-k results. We assume the scoring function is supplied by the upstream agent. DASE is agnostic to its origin; in our own research agent, the weights are trained

using logistic regression over a *calibration set* [38] of labeled samples. These assumptions are consistent with Fagin's monotonicity assumption [14] and thus admit efficient threshold-based top-k evaluation. DASE then outputs a *ranked list*. The upstream agent might use this in either of two ways:

1. Direct consumption. When the agent's question reduces to finding the best-matching combinations of evidence—a typical pattern in wide search and multi-objective discovery—the ranked output is the answer. Cross-encoders [37] are a common way for the agent to map per-source signals into a unified ranking. For instance, they may provide a ranking for candidate pharmaceuticals based on desirable properties, such as oral bioavailability and level of toxicity.

2. Prefilter for semantic operators. Recent data processing engines with semantic operators [38, 43, 47] use LLMs for judging whether tuples (and, for joins, tuple combinations) satisfy conditions. Our methods can also be used in this scenario, providing high-recall, prefiltered select-project-join candidates (computed over embedding distances and structured elements) before expensive LLM semantic evaluation [38] is performed.

In either usage scenario, DASE executes the agent-supplied top-k scoring function over structured attributes, multiple embedding spaces, and cross-table joins. We now formalize this query.

2.1.1 Query Model. A multistep reasoning query Q is formally defined by the triple $(\mathcal{P}, \mathcal{S}, \mathcal{J})$, which characterizes the feasibility space, ranking objectives, and relational dependencies.

Predicates ($\mathcal{P}$): $\mathcal{P} = \{p_1, \ldots, p_{|\mathcal{P}|}\}$ is a set of predicates over 1 or more **structured components** (which may be the results of a join or Cartesian product). This set may be empty ($\mathcal{P} = \emptyset$), indicating an unconstrained search space. Regardless of their specific semantic types (such as range constraints, set memberships, or probabilistic filters) each $p_i \in \mathcal{P}$ can be conceptually modeled as a Boolean indicator function $p_i : \mathcal{R} \to \{0, 1\}$. For a given query subexpression q and any resulting row instance $r \in q(\cdot)$, $p_i(r)$ logically evaluates to 1 if r satisfies the constraint, and 0 otherwise.

Similarity Join Conditions ($\mathcal{J}$): $\mathcal{J} = \{(e_{1,i}, e_{2,i}, dis_i, \tau_i)\}_{i=1}^{|\mathcal{J}|}$ is an optional set of relational constraints ($\mathcal{J} = \emptyset$ denotes a standard single-table query). Each condition specifies a thresholded embedding-similarity join between **unstructured components** across tables: a link is established when the embedding distance satisfies $dis_i(e_{1,i}, e_{2,i}) \leq \tau_i$. Thus, distances are lower-is-better feasibility predicates, not ranking scores.

Scoring Model ($\mathcal{S}$): $\mathcal{S} = \text{score}_f(s_1, \ldots, s_{|\mathcal{S}|})$ defines the unified ranking objective. To guarantee top-k retrieval, the query must specify at least one scoring signal ($|\mathcal{S}| \geq 1$). Every s_i follows *higher-is-better affinity*: an embedding distance d is converted by a calibrated non-increasing transform ϕ_i, e.g., $s_i = \phi_i(d) = -d$; and a scalar attribute is analogously mapped to a desirability score. Score components include (1) query-to-object affinity $\phi_i(dis(\mathbf{q}_v, e_i))$, (2) relational-link affinity $\phi_i(dis(e_{1,i}, e_{2,i}))$, and (3) structural-attribute desirability. The aggregation function f is coordinatewise nondecreasing (for example, a nonnegative weighted sum), so higher component affinities cannot lower the final score. The goal is to retrieve the top-k tuples that satisfy $\mathcal{P}$ and $\mathcal{J}$ while maximizing $\mathcal{S}$.

As discussed above, the scoring signals and weights are provided by an upper-layer agentic system (e.g., through calibration or learned rankings [9, 29, 61]) or by direct user specification; our methods are agnostic to their origin, provided the resulting component scores obey the higher-is-better, monotone contract above.

```
SELECT m1.molecule_id, m2.molecule_id, d2.pubmed_id,
       semantic_score(m1, p2, $molecule_embed) AS
    semantic_score
FROM molecule m1
    JOIN paragraph p1 ON m1.paragraph_id = p1.
     paragraph_id
    JOIN paragraph p2 ON p2.text_embed <=> m1.
     description_embed < 0.25
    JOIN document d2 ON p2.pubmed_id = d2.pubmed_id
    JOIN paragraph p3 ON d2.pubmed_id = p3.pubmed_id
    JOIN molecule m2 ON p3.paragraph_id = m2.paragraph_id
WHERE  m1.chemberta_embed <=> $molecule_embed < 0.3
    AND p1.document_id <> d2.pubmed_id
    AND m1.molecule_id <> m2.molecule_id
ORDER BY semantic_score DESC LIMIT 100;
```

Listing 1: Abbreviated molecular-relationship query pattern for `$molecule_embed`, for Example 1.2.

Our task extends classic rank aggregation [14, 48]. Traditional settings find top answers by aggregating *independent* scores that evaluate the query against individual records. In contrast, our core challenge is computing dynamic, *interdependent* scores to evaluate embedding-similarity join conditions on the fly. Reconciling these highly coupled data-to-data affinities with structured logical predicates, while avoiding exhaustive search and leveraging indexing, is the primary computational hurdle in retrieving the top-k results.

Example 2.1. Listing 1 gives an abbreviated SQL query for Example 1.2. The `semantic_score` function computes a higher-is-better result, by negating the calibrated weighted combination of the lookup and join distances.

Standard ANN indices fall short here for two reasons. Their current implementations in many DBMS engines (such as PostgreSQL) are designed for 1-to-N point lookups, making them **unable to execute M-to-N embedding-similarity joins following a block nested loops pattern**. Instead, they are limited to tuple nested loops joins. Second, because they **cannot jointly evaluate the multi-signal scoring function**, the true top results for the overall query might be buried arbitrarily deep within the ANN's ranked stream, precluding early termination.

The M-to-N embedding-similarity pattern is integral to many object-linking tasks, such as record linking [11] or textual similarity detection. In many cases, we will have a block of M items from the left relation (collection) that are highly similar to N items in the right relation; the final answer set may be ranked with a holistic combination of score components (the multi-signal scoring function) including the similarities of each similarity step, as well as the intrinsic scores of the original items versus specific query constraints. What is needed is an efficient way to focus computation on the most promising sets of candidate answers (across multiple similarity joins), and to apply filter predicates and ranking functions to these candidates. If we could decouple join discovery from scoring, we could support threshold algorithms [14] for earlier termination.

2.2 Semantic Joinability Is a Tail Event

Naively evaluating an embedding-similarity join requires computing distances over a Cartesian product of endpoint embedding pairs. In high-dimensional, approximately isotropic embedding spaces[1], pairwise distances concentrate in a dense bulk, while unusually close pairs occupy a small left tail [49]. The join candidates with semantic affinity are *sparse* relative to all pairs. Figure 3 illustrates this empirical pattern for IMDB actor–director embeddings: most distances lie between 0.55 and 0.85, with a small close-pair tail.

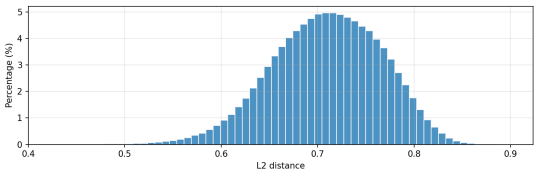


Figure 3: L2 distance distribution: pairwise among IMDB Actor_Director embeddings (Gemini-embedding-001)

This observation suggests **it is scalable to materialize the close-pair tail** as a **sparse embedding-similarity join index** and feed its endpoints to batched retrieval. (Even when the joinable candidates do not follow the Gaussian distribution, the tail property generally still holds; we discuss this in Section 3.3.)

3 Semantic Join Index

Based on the intuition above, the **Semantic Join Index (SemJI)** materializes close vector pairs, instantiating the embedding-similarity-join topology $\mathcal{J}$ of our query model. Here "semantic" refers to the embedding-based proxy; it is distinct from a downstream LLM semantic operator [38].

Definition 3.1. Pairwise SemJI. Given a threshold τ_{cap}, a pairwise SemJI with **index-coverage boundary** τ_{cap} is the relation:

$$\mathcal{J}_{\tau_{cap}} = \{(id_A, tid_A, id_B, tid_B, \text{dist}(id_A, id_B)) \mid \text{dist}(id_A, id_B) \leq \tau_{cap}\}$$

where A, B are tuples; id_A, id_B are values of the fields whose distance is being indexed; tid_A, tid_B are tuple pointers; and $\text{dist}(id_A, id_B)$ is the pairwise value distance. The distance is converted to the query's higher-is-better affinity during ranking.

An exact SemJI construction stores every pair in $\mathcal{J}_{\tau_{cap}}$. For a pairwise similarity query with threshold $\tau_q \leq \tau_{cap}$, the index answers its similarity-join predicate exactly. For $\tau_q > \tau_{cap}$, the SemJI index is insufficient, and we must additionally enumerate the *outer band* $\tau_{cap} < \text{dist}(A, B) \leq \tau_q$ through standard methods, by querying against the nearest-neighbor indices.

Intuitively, we can compositionally evaluate multi-hop paths (e.g., $A \xrightarrow{\tau_1} B \xrightarrow{\tau_2} C$) using a SemJI, by treating its materialized connectivity as a relational view. For a set of similarity predicates $\mathcal{J}_{chain} = \{(\mathcal{J}_1, \tau_1), \ldots, (\mathcal{J}_m, \tau_m)\}$, the engine computes:

$$\mathcal{R}_{chain} \leftarrow Fetch(\mathcal{J}_1, \tau_1) \bowtie Fetch(\mathcal{J}_2, \tau_2) \bowtie \ldots \bowtie Fetch(\mathcal{J}_m, \tau_m) \quad (1)$$

where each *Fetch* applies its runtime threshold $\tau_i \leq \tau_{cap}$. We describe this process in Section 4. In the remainder of this section, we present a practical implementation of the SemJI; then we consider how to set and adjust τ_{cap}, and how to address anisotropism in the distance measure using multi-way SemJI structures.

[1]"Approximately isotropic" means that variance is not concentrated in a small number of embedding dimensions.

3.1 The SemJI Index

In practice, we seldom have access to a full enumeration of nearest-neighbor pairs, since exact computation and indexing are prohibitive at scale. Instead, we will leverage approximate nearest neighbor (ANN) indices. An ANN-built SemJI stores $\widehat{\mathcal{J}}_{\tau_{cap}} \subseteq \mathcal{J}_{\tau_{cap}}$. Filtering by distance removes false positives, but ANN traversal can miss an under-the-cap pair, yielding false negatives. A query result using an approximate SemJI is exact only with respect to its retrieved candidate set. Our work leverages HNSW [35], one of the most widely adopted ANN algorithms, although it has no general completeness guarantee. Our techniques generalize to other hierarchical graph-based ANN strategies [18].

Radius-expanded HNSW (Algorithm 1). Standard HNSW is optimized for top-k retrieval and can prune candidates inside a requested radius. Our **radius-expanded traversal** relaxes the pruning gate to the maximum of the beam boundary and τ_{cap}: $\text{gate} = \max(\mathcal{W}.\text{furthest}(), \tau_{cap})$. This improves empirical range-query recall, although it does not establish completeness.

Algorithm 1 Layer Traversal in Radius-Expanded HNSW

Require: Query q, Entry Point ep, Beam ef, Query Radius $\tau_q \leq \tau_{cap}$, Construction Cap τ_{cap}
Ensure: Approximate Result Set $\mathcal{R}$

$d_{ep} \leftarrow \text{dist}(ep, q)$; $C \leftarrow \text{MinHeap}(ep, d_{ep})$; $\mathcal{W} \leftarrow \text{MaxHeap}(ep, d_{ep})$; $\mathcal{V} \leftarrow \{ep\}$
$\mathcal{R} \leftarrow \emptyset$
if $d_{ep} \leq \tau_q$ **then** $\mathcal{R}$.add(ep)
end if
while C is not empty **do**
  $c \leftarrow C$.pop(); $w \leftarrow \mathcal{W}$.peek()
  if $c.dist > w.dist$ **and** $c.dist > \tau_{cap}$ **then**
    break ▷ Prune only if outside both beam and radius
  end if
  $gate \leftarrow \max(\mathcal{W}.\text{peek}().dist, \tau_{cap})$
  for $e \in \text{neighbors}(c)$ **do**
    if $e \notin \mathcal{V}$ **then**
      $\mathcal{V}$.add(e); $d_e \leftarrow \text{dist}(e, q)$
      if $d_e \leq gate$ **or** $|\mathcal{W}| < ef$ **then**
        C.push(e, d_e); $\mathcal{W}$.push(e, d_e)
        if $|\mathcal{W}| > ef$ **then** $\mathcal{W}$.pop_furthest()
        end if
        if $d_e \leq \tau_q$ **then** $\mathcal{R}$.add(e)
        end if
      end if
    end if
  end for
end while
return $\mathcal{R}$

Batched Execution Optimization. A key optimization we make is to batch ANN index probes. We group HNSW probes with shared upper-layer traversals.

Space Cost. SemJI is a materialized relation, with $J = |\mathcal{J}_{\tau_{cap}}|$ retained pairs. Its base representation and any distance/order or endpoint lookup indexes require $O(J)$ records. Because the SemJI is constructed only for frequently used embedding-column pairs whose calibrated tail has a bounded expected degree, the retained-pair fraction $\rho = J/(|A||B|)$ is small, so storage scales as $\rho|A||B|$ rather than the full product. Nonetheless, we enforce a maximum space budget per index, adjusting τ_{cap} to reach that budget. The configurations in Section 5 show practical settings.

3.2 Setting and Updating the SemJI Threshold

A space budget B_{max} upper-bounds every SemJI. The budget is over retained pairs, not on the numerical radius: embedding geometry can make a small change in radius admit many pairs, whereas J is directly proportional to base SemJI storage. Even with a large budget, we materialize a very small percentage of the possible pairs, as we demonstrate in Section 5.

Initially, we run a small warmup sample and query, and estimate the τ_{cap} required to answer the query, also requiring that the SemJI cardinality remain below the limit of B_{max}.

Adaptive expansion. The workload may introduce queries that require a higher threshold. At query time, we ensure correctness by unioning the query with answers from a separate *outer band* query over the base HNSW index, as described in the previous subsection.

Offline, the controller grows the index by its *materialized-pair capacity*, which allows us to amortize expansion of the cap. Let $J(\tau) = |\mathcal{J}_\tau|$ and let $J = J(\tau_{cap})$ be the current number of retained pairs. The controller records the overflow and asynchronously doubles its retained-pair capacity, targeting B' pairs at radius τ_{next},

$$B' = \min\{B_{max}, \max\{2J, J_{min}\}\}, \quad \tau_{next} = \inf\{\tau \geq \tau_{cap} : J(\tau) \geq B'\},$$

where J_{min} prevents a near-empty initial index from growing in negligible increments. The controller expands only when $J < B_{max}$, and inserts the new values in the range $(\tau_{cap}, \tau_{next}]$.

Amortized cost. For a static relation, capacity doubling yields at most $\lceil \log_2(B_{max}/J_{min}) \rceil$ expansions. If an expansion discovers, writes, and indexes only new distance-bands, the total pair-materialization work through final size J_f is $O(J_f - J_0)$. Even rebuilding the materialized relation at every doubling has a geometric-series bound below $2J_f$ for pair writing. This is an output-space amortization guarantee, where the cost of the rescan is dependent on the radius-expanded HNSW of Algorithm 1.

A single unusually permissive query need not make the index cover τ_q: strict capacity doubling preserves the budget, while that query still uses its outer-band fallback. Our expansion policy is also independent of the distribution of pairs under the threshold, although anisotropy can change the radius needed to admit B' pairs.

3.3 Robustness to Correlated Embeddings

SemJI sets a materialization radius based on the pairwise-distance population being sufficiently isotropic for a tail threshold such as $\tau_{cap} = \mu - 3\sigma$ to be informative. This can fail when endpoint embeddings are anisotropic or share dominant directions, e.g., because they were extracted from overlapping source context and their distributions are correlated. Such geometry can distort the apparent distance distribution, so a statistically chosen materialization radius may omit useful close pairs [13, 36, 46].

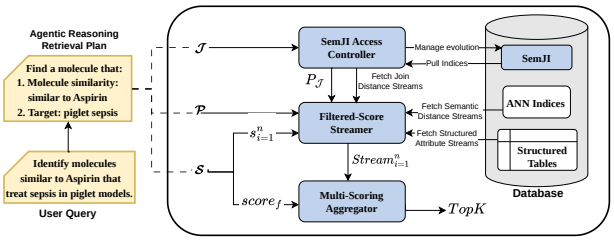


Figure 4: DASE architecture

We can detect anisotropy from the covariance of the centered embeddings from a query; if the variance is concentrated in a few directions, embedding distance thresholds may not form a tight ball around the query-relevant multi-hop neighbors. Here, we adopt mitigation strategies from the literature for anisotropic data. **PCA whitening** mean-centers and rescales principal axes [46]; **all-but-the-top (ABTT)** removes a configured number of dominant directions [36]. Because these transformations define different distance metrics, SemJI builds a matching index and recalibrates τ_{cap}.

For multistep distances where anisotropy and correlation mean the multi-way distance is not easily estimatable from pairwise distances, we can generalize to **multi-way SemJI structures**: relations capturing n-ary relationships and calibrated distances across sets of correlated nodes, akin to Access Support Relations [23].

Even when anisotropy makes the choice of the index-selection threshold more challenging, it typically does not make the SemJI dense. SemJI materializes pairs with $\text{dist}(a, b) \leq \tau_{cap}$; equivalently, τ_{cap} retains pairs whose distance-derived affinity is at least the calibrated threshold. As Section 5.7 shows, these near-neighbor edges remain a tail of the real endpoint-pair products (0.03–1.09% under the evaluated policies), even for the anisotropic corpora.

4 DASE Execution Strategy

The execution layer of DASE (Figure 4) consists of three core components that interoperate to tackle multistep reasoning with both similarity and Boolean predicates: the **Multi-Scoring Aggregator** (MSA) drives top-k evaluation of results according to the score function, multiple **Filtered-Score Streamers** (FSSs) apply filter predicates, and the **SemJI Access Controller** leverages SemJI connectivity. Predicate-aware ANN traversal is one access mode within FSS, alongside predicate-first and vector-first execution. Our approach requires a coordinatewise non-decreasing scoring function over higher-is-better component scores [14]. Before we describe those three components, we first introduce an extension to ANN (HNSW) index support within the query engine.

4.1 Predicate-aware ANN Index Traversal

Most hybrid search systems split their strategies into two camps [18, 33]: *vector-first* (post-filtering) and *filter-first* (pre-filtering). This creates a performance gap: vector-first works well when most items match the filter, while filter-first is better when the filter eliminates almost everything. Between these two extremes lies a "middle ground" where neither approach is efficient. To handle this, we implemented a **Predicate-aware ANN Index Traversal** that checks filters while walking the index graph. **Often one of the filter**

conditions is whether the ANN node endpoints lie within a batch of path endpoints retrieved from the SemJI index.

Simulating an Oracle Graph. Our method is inspired by the *Oracle Graph* idea from ACORN [39]. Conceptually, if we had a custom index built only from the items that pass the filter, search would be optimal. Since we cannot build a new index for every query, we simulate this experience using the global HNSW index.

Transitive neighbor expansion. The main problem with walking a global ANN graph under constraints is that a path to a good candidate may need to traverse a node that fails the filter. To mitigate this, we modify the greedy search: When the immediate neighbors of a node are all filtered out, we look at their neighbors (2 hops away). By using these invalid nodes as bridges rather than dead ends, we can jump over gaps in the graph and find the true nearest neighbors that satisfy the query.

Bloom filter optimization. As noted above, DASE will frequently combine SemJI results with an ANN traversal — doing a similarity join over a similarity-based lookup. Checking the filter condition for every node during a graph traversal is too slow if it requires fetching data from the main table. To speed this up, we compile the query's node filter set into a **Bloom filter** [4] before the search begins. This allows the traversal engine to check if a node is valid using a fast bitwise operation. For a target false-positive rate ϵ, the Bloom filter uses $O(N \log(1/\epsilon))$ bits and takes $O(N)$ time to build. If the dataset is too large, the system falls back to standard disk-based strategies to avoid exhausting memory.

4.2 Filtered-Score Streamer

The **Filtered-Score Streamer (FSS)** generates a sorted stream for **one** higher-is-better scoring signal under constraints. Formally, its input is (1) a set of predicates $\mathcal{P}$ and (2) a scoring signal s, which is one of: (a) query-to-object affinity, (b) relational-link affinity, or (c) structural-attribute desirability, according to the Query Model in Section 2. Distance-derived signals are first transformed by the query's non-increasing affinity function. The output is an iterable stream of entries that pass the filters in descending score order: $FSS(\mathcal{P}, s) \rightarrow Stream((id_i, score_i))$. The FSS adapts its execution path based on the data source and predicate selectivity σ.

(a) Semantic Path: For a query-to-object affinity derived from $\text{dist}(\mathbf{q}, e_i)$, the scorer selects between attribute-first filtering, vector-first ANN traversal, and the predicate-aware ANN traversal based on σ to minimize distance-computation overhead. When $\sigma < \sigma_1$, attribute-first filtering leaves a short candidate list for direct distance computation; when $\sigma > \sigma_2$, vector-first search with expanded retrieval and post-filtering is preferable; and when $\sigma_1 < \sigma < \sigma_2$, the system uses predicate-aware traversal.

Selection of σ thresholds. Offline profiling identifies crossover selectivities σ_1, σ_2 at a fixed recall target: attribute-first below σ_1, vector-first above σ_2, and predicate-aware ANN between them.

(b) Relational Path: For a relational-link affinity derived from $\text{dist}(A, B)$, the scorer scans the **join index** in ascending distance order (equivalently, descending affinity order), while applying $\mathcal{P}$ to filtering attributes stored in the join index or entity metadata. This can be accelerated by traditional B-tree indices on the join index.

(c) Attribute Path: For a structured attribute scoring signal, we perform filtering with B-tree indices over the relational data.

4.3 Multi-Scoring Aggregator

The **Multi-Scoring Aggregator (MSA)** combines the sorted streams $\{\mathcal{S}_i\}_{i=1}^n$ produced by FSS under a coordinatewise non-decreasing, higher-is-better aggregation function $score_f$, returning $\text{MSA}(\{\mathcal{S}_i\}_{i=1}^n, score_f, K)$. It is a direct adaptation of Fagin's Threshold Algorithm (TA) [14], with Rank-Join-style random access [28] to reconstruct relationally valid tuples. Thus, MSA does not replace TA's ranking logic: it retains sorted access, random access, a top-k buffer, and TA's threshold stopping test. Note that standard TA aggregates multiple scores for the same entity; it does not apply to our cross-table setting, where scores are jointly evaluated across entities in multiple tables.

On observing an entry e from any stream, MSA uses SemJI lookups and sibling attributes to materialize every valid tuple containing e, scores each newly discovered tuple, and maintains the best K. The only DASE-specific changes are that random access can traverse a multi-hop SemJI predicate and that the next stream need not be selected round-robin. If $L[i]$ is the most recently seen score of stream i, every unmaterialized tuple lies below every current cursor and is bounded by $\tau = score_f(L[1], \ldots, L[n])$. MSA stops when its current K-th score is at least τ.

Exact Answers. Following the usual TA guarantee, the result is exact when all streams are complete and globally sorted, random access reconstructs every valid tuple containing a seen entry, and component scores are exact. If we leverage ANN traversal or an approximate SemJI, the stopping rule provides exact answers with respect to the set of retrieved candidates.

Distribution-Aware Stream Selection. TA normally advances streams in round-robin order. MSA instead chooses the stream expected to reduce the termination gap $G = \tau - \theta_K$ most, where θ_K is the current K-th score. For a linear $score_f = \sum_i w_i s_i$, profiling supplies each stream's local score density f_i (an empirical density or a fitted Gaussian). The expected threshold reduction from advancing stream i is approximately $w_i/(N f_i(L[i]))$. MSA also estimates the expected increase in θ_K from the newly materialized tuples using the other streams' score distributions, and advances $i^* = \arg\max_i \{\mathbb{E}[\Delta\tau_i] + \mathbb{E}[\Delta\theta_{K,i}]\}$. Density and truncated-expectation values are tabulated during profiling. This is a greedy scheduling policy layered on TA, without changing its stopping condition or exactness guarantees. Its candidate-elevation estimate uses marginal stream distributions, and is therefore heuristic when ranking signals are correlated.

4.4 SemJI Access Controller

The **SemJI Access Controller** functions as the storage abstraction layer that exposes the materialized semantic connectivity of SemJI to the query pipeline. It serves two distinct query-time roles depending on whether the relational data is treated as a hard constraint or a ranking signal. Additionally, it acts as the runtime monitor that drives the structural evolution of the index.

Materializing join predicates. When the query includes hard relational constraints ($\mathcal{J}$), the controller's objective is to define the valid search space. It retrieves the set of materialized pairs for each join condition and, in the case of multi-hop dependencies, joins these sets to synthesize a unified **candidate set** of valid tuples. This process effectively converts topological constraints into a flat filter

predicate $P_{\mathcal{J}}$, which is pushed down to the *Filtered-Score Streamer* to prune the search space. Symbolically, $P_{\mathcal{J}} \leftarrow\!\bowtie_{i=1}^{|\mathcal{J}|} \text{SemJIFetch}(\mathcal{J}_i)$.

Streaming relational scores. When the query utilizes relational proximity as a ranking objective (i.e., a scoring component $s \in \mathcal{S}$ is defined by the semantic link distance), the controller acts as a streaming source. It scans the underlying index to yield pairs (id, dis) sorted by their precomputed semantic distance. This stream is fed directly into the *Multi-Scoring Aggregator* as a standard ranking signal: $\mathcal{S}_{rel} \leftarrow \text{SemJIFetch}(\mathcal{J}_{\mathcal{S}})$.

Workload monitoring and evolution trigger. As described in Section 3.2, the controller performs workload monitoring. During query execution, it tracks the *base-engine completion rate*: the fraction of queries whose materialized SemJI coverage is exhausted, i.e., that overflow to $\tau_q > \tau_{cap}$. If this rate consistently exceeds its tolerance, the controller asynchronously triggers a capacity-doubling expansion, growing $\tau_{cap} \rightarrow \tau_{next}$ within the budget B_{max} via Radius-Expanded HNSW. This allows the index to evolve without bottlenecking real-time execution.

5 Experiments

Datasets. To evaluate DASE on deep research reasoning tasks, we constructed two benchmark datasets. In **IMDB**, base tables from the official IMDB (https://datasets.imdbws.com) are augmented with structured fields from the repository `jquigl/imdb-genres`. To support cross-table movie comparisons, the data is split by release year. In **MOLECULE**, we collected query templates from our agentic AI system built on a PubMed-derived molecule database (https://pubmed.ncbi.nlm.nih.gov/). These templates are used to simulate multistep reasoning workflows (e.g., discovering candidate molecules by joining fragmented evidence). The dataset consists of Papers, Facts, and Molecules. Benchmark and workload details are available in our technical report (https://anonymous.4open.science/r/DASE-2C61/).

We include two public benchmark datasets that allow us to examine specific aspects of DASE. The first of these is **SemBench** [27], which we use to explore the effectiveness of DASE in prefiltering for semantic queries. Our second benchmark is **NFCorpus** [5], a 2,868-paper biomedical corpus with title embeddings, real intervention–condition and study–outcome links, and categorical attributes. Its depth-scaling workload ranges from two to eight join steps, providing diverse multistep reasoning queries at different cardinalities.

Workloads. We developed synthetic workloads over our two datasets, following a taxonomy shown in Table 2 and fully detailed in our technical report. These cover all possible combinations of structured predicates ($\mathcal{P}$), multi-component scoring ($\mathcal{S}$), and cross-table joins ($\mathcal{J}$). In this framework, single-vector similarity search serves as the default configuration. The component $\mathcal{S}$ represents the transition from simple embedding retrieval to a generalized scoring function $f(s_1, \ldots, s_n)$. This allows the system to aggregate multiple vector similarities, as required for complex discovery workflows.

The eight classes cover these requirements separately and in combination: W1 is the single-signal baseline; W2, W3, and W5 introduce filtering, multiple scores, and joins, respectively; and W4, W6, W7, and W8 combine them. The FSS/MSA expressions in Table 2 illustrate their execution in DASE, rather than prescribing how other systems must execute these logical workloads.

Additional workloads are from the published SemBench and NFCorpus benchmark specifications.

Implementation strategies. We compare DASE against five execution strategies and DBMSs:

(1) **PostgreSQL:** The multistep reasoning query is submitted directly to the PostgreSQL engine. The embedding columns are indexed using `pgvector`'s HNSW or IVFFlat structures. All execution decisions, including join order, index selection, and predicate pushdown, are delegated to the PostgreSQL optimizer.
(2) **Filter-First:** Evaluates structural and exact-match predicates first (using the `WHERE` clause) to generate a candidate set of valid IDs. The system then performs random access to fetch the corresponding embeddings, computes the similarity scores on-the-fly, and maintains a Top-k heap to produce the result.
(3) **Score-First:** An inverted strategy that prioritizes similarity search. It uses an `ORDER BY` clause to retrieve a continuous stream of records sorted by their joint similarity score. The system applies the structural predicates as a post-filter and halts the stream immediately once k valid entities are identified.
(4) **Rerank:** For queries involving multiple scoring signals, executes a retrieve-and-rerank pipeline. It streams candidates separately from each signal's index, applies predicates as post-filters, and dynamically evaluates the joint score to update the Top-k results until the exploration stopping criteria are met. For single-signal queries, it gracefully degrades to Score-First.
(5) **Milvus:** Milvus [50] is a state-of-the-art vector database. For single-signal queries (with or without structural predicates), we push down the filters directly into Milvus. Because Milvus lacks support for multi-signal scoring or relational joins, queries involving these operations are streamed to the application layer, which applies the Rerank strategy to finalize the Top-k results.

5.1 Setup and Methodology

DASE is implemented in about 10,000 lines of Python and C, with part of the development driven by Claude Code (Opus 4.6–4.8) and extensive manual code review. Experiments were conducted on PostgreSQL 18.3 with the DASE-modified pgvector extension (v0.8.5). Text vectors use `Gemini-embedding-001` (1536 dimensions); MOLECULE additionally includes molecular descriptors and an ECFP signal. We evaluated a set of multistep reasoning queries involving structured filters, multi-vector relevance scoring, and join-based conditions. All vector columns use L2 HNSW indexes with $m = 16$, $ef_{construction} = 64$, and default $ef_{search} = 40$. Each database instance runs on a dedicated set of 16 physical AMD EPYC 9575F cores (up to 5.0 GHz) and 64 GB of RAM. This hardware supports AVX-512 instructions, which pgvector leverages to accelerate vector distance computations. Evaluation was exclusively CPU-based.

We report each method as a recall–throughput curve. For every method we sweep a search-effort parameter and record recall against QPS: the HNSW ef_{search} and IVFFlat probe count for index scans, the threshold-algorithm admission tolerance ε for DASE and Rerank, the post-filter width for Milvus, and a per-query time budget for best-effort strategies. Each operating point averages 5 runs after discarding a warm-up run, and shaded bands denote ±1 standard deviation; per method we retain the Pareto frontier over the resulting (recall, QPS) points. Recall is measured against the

Table 1: Main experimental data and configuration. SemJI counts are the materialized-pair records in the released configurations; a workload selects the radius appropriate to its join threshold.

Scenario	Base relations	Candidate Links / SemJI pairs	Workload (queries/class, top-20)
IMDB	`imdb_t1`: 42,378 rows; `imdb_t2`: 43,237 rows	τ_{cap} = 0.5: 2.10M; 0.6: 58.41M pairs	W1–W8; 84–100
MOLECULE	`facts_50k`: 50,000 rows; `paper`: 62,857 rows	τ_{cap} = 0.5: 0.032M; 0.6: 1.38M; 0.7: 63.48M pairs	W1–W8; 93–100
SemBench	Movie, E-Commerce, Wildlife, MMQA, and Cars scenarios	Calibrated embedding candidate links for semantic joins; no shared deep-research SemJI	F/L/J/M/C/R tasks; DASE prefilter and DASE + BigQuery
NFCorpus	2,868 papers; title embeddings	τ = 0.35: 21,466 intervention–condition; 254 study–outcome pairs	depth 2–8; 90 instances (3 × 30)

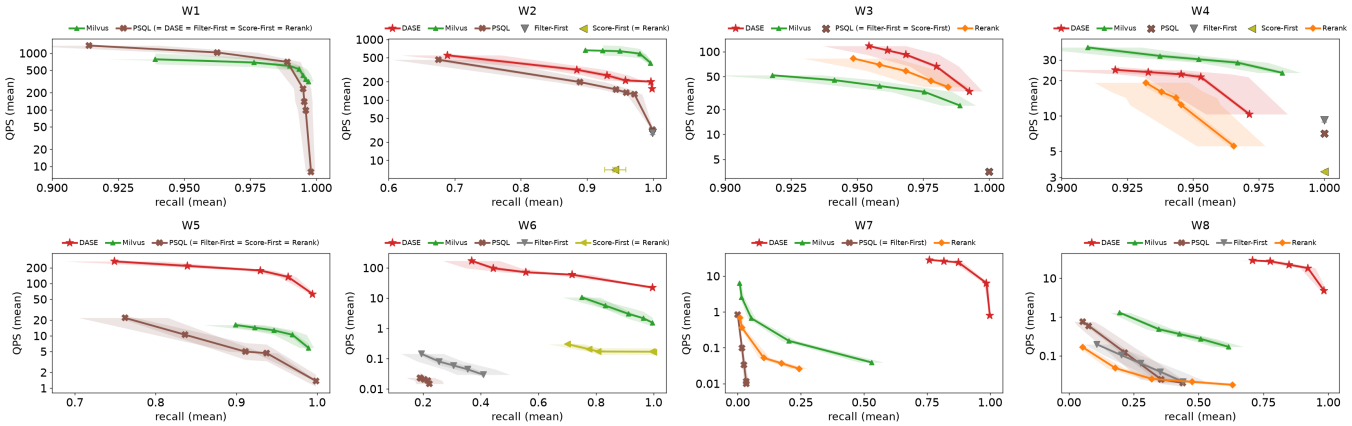


Figure 5: Multistep Reasoning Top-k: Deep Research workload on IMDB

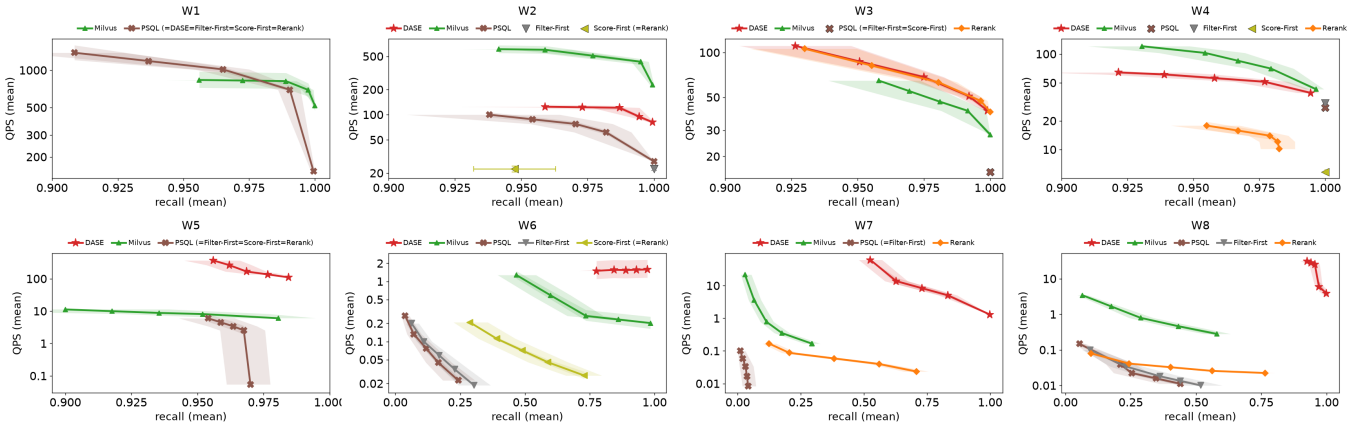


Figure 6: Multistep Reasoning Top-k: Deep Research workload on MOLECULE

released exhaustive top-20 result for each planned query. Index construction time is isolated and excluded from all QPS measurements. Table 1 reports the dataset-specific relation, SemJI, and workload configurations; query-specific thresholds and score weights are stored with the released workload instances.

For SemBench, **DASE** performs ranked embedding-based prefiltering, omitting LLM judgment, using counterfactual anchors for semantic filters and calibrated thresholds for semantic joins. **DASE + BigQuery** sends the same candidates to BigQuery semantic operators; its cost and latency therefore describe the composed pipeline. Table 3 reports USD, the SemBench task metric, and end-to-end wall-clock latency including remote Gemini API time; `X` denotes unsupported configurations and scenario averages exclude them. The LOTUS, Palimpzest, ThalamusDB, and BigQuery columns reproduce SemBench's published `gemini-2.5-flash` configuration [27]; only the DASE and DASE + BigQuery columns are ours.

5.2 Multistep Top-k Reasoning

Deep Research. We evaluate DASE and the alternatives across the **IMDB** (Figure 5) and **MOLECULE** (Figure 6) datasets over our 8 workloads. Each panel plots the recall–throughput (QPS) Pareto frontier obtained by sweeping each method's search-effort as introduced in the setup section (Section 5.1); up and to the right is better. Milvus excels on the single-signal W1 workload, but its advantage reverses when queries require multi-score ranking. Filter-first becomes a sequential scan without predicates, rerank reduces to score-first with one signal, and score-first is infeasible for W7/W8

Table 2: Characterization of query workload classes.

WL	$\mathcal{P}$	Multi-$\mathcal{S}$	$\mathcal{J}$	Pattern	Expression
Intra-Entity Retrieval					
W_1	×	×	×	Semantic retrieval	$TopK\{FSS(\emptyset, s)\}$
W_2	✓	×	×	Filtered retrieval	$TopK\{FSS(\mathcal{P}, s)\}$
W_3	×	✓	×	Multi-scoring retrieval	$MSA(\{FSS(\emptyset, s_i)\}_{i=1}^n, score_f, K)$
W_4	✓	✓	×	Filtered multi-scoring	$MSA(\{FSS(\mathcal{P}, s_i)\}_{i=1}^n, score_f, K)$
Inter-Entity Synthesis					
W_5	×	×	✓	Semantic join	$TopK\{FSS(P_{\mathcal{J}}, s)\}$
W_6	✓	×	✓	Filtered join	$TopK\{FSS(\mathcal{P} \cup P_{\mathcal{J}}, s)\}$
W_7	×	✓	✓	Multi-scoring join	$MSA(\{FSS(P_{\mathcal{J}}, s_i)\}_{i=1}^n, score_f, K)$
W_8	✓	✓	✓	Filtered multi-scoring join	$MSA(\{FSS(\mathcal{P} \cup P_{\mathcal{J}}, s_i)\}_{i=1}^n, score_f, K)$

because it forms a full Cartesian product. On W3, DASE outperforms round-robin rerank, indicating the value of adaptive stream selection; across W5–W8, DASE and SemJI sustain higher QPS at matched recall. We separately study how performance scales with the number of coupled join positions in Section 5.4.

Semantic queries. Table 3 summarizes cost, quality, and latency on SemBench. DASE alone suffices for semantic-filter (**F**) tasks that reduce to a binary or coarse distinction, such as positive vs. negative reviews in *Movie*: comparing embedding distance to counterfactual anchors returns valid answers at orders-of-magnitude lower cost and latency than any LLM-backed baseline. For semantic-map (**M**) and LLM-semantic-join (**J**) tasks that exceed embedding-distance reasoning, DASE + BigQuery substantially raises quality (e.g., *MMQA* avg. 0.11 → 0.76). Prefiltering with DASE also lets BigQuery beat its own standalone numbers at a fraction of the cost: on *E-Commerce*, \$0.54/38s at quality 0.80 versus BigQuery alone at \$2.42/45s and 0.67. The LLM sees only the prefilter's survivors, so it spends less and decides better. The one failure mode is embedding quality itself: in *Wildlife*, rare words like "impala" are poorly aligned, so neither configuration recovers fully.

Overall, DASE delivers low-cost answers when embedding distance suffices, and a high-recall prefilter that makes downstream LLM evaluation more efficient and accurate when it does not.

5.3 SemJI Construction and Maintenance

We build SemJI using HNSW-based ANN range queries with a radius threshold. Table 4 shows an ablation from a naive iterative approach, through a batched lateral join, to a fully vectorized whole-table HNSW range query — a single `JOIN LATERAL` that runs entirely in-database. This reduces construction time from 81.2 hours to 1.6 minutes, only losing 2 percentage points in recall.

SemJI evolution. We evaluate the capacity-based maintenance policy (Section 3.2) on the linkage-only MOLECULE W5 stream of 2,000 seed queries. A random warmup sample fixes $\tau_{init} = \mu - 3\sigma = 0.607$ before the workload is observed; overflowing queries use the base-engine outer-band fallback while the controller grows the index asynchronously.

With this stream, two capacity doublings grow SemJI from 83K to 333K pairs and reach $\tau_{converged} = 0.6526$ (Figure 7). The final 333K retained pairs are only 0.011% of the 3.14B fact–paper Cartesian product (6.7 links/fact and 5.3 links/paper on average), so SemJI remains highly compact relative to exact join enumeration. The held-out overflow rate falls from 80.6% to 49.8%; the remaining overflows are budget-limited sparse seeds. Ascending and random query orders reach the same final radius (spread < 0.001) with two expansions, only differing in when the work is paid (a 1.14× cumulative-latency difference).

5.4 Scaling Multistep Reasoning Depth

We isolate how each system scales vs. the number of coupled join positions k, on the multi-dimensional NFCorpus workload $Q = (P, S, J)$. Here a k-step query fixes per-position anchors $a_1, \ldots, a_k$ and asks for the top-n length-k walks $(p_1, \ldots, p_k)$ satisfying

$$\min \frac{1}{k} \sum_{i=1}^{k} \mathrm{dist}(a_i, p_i) \quad \text{s.t.} P(p_i)\ \forall i, \mathrm{dist}(p_i, p_{i+1}) \le \tau\ \forall i, \quad p_i \neq p_{i-2},$$

i.e. a monotone score S (title-embedding distance) minimized subject to a predicate P (selectivity σ=0.18) and a threshold intervention–condition similarity join J (τ=0.35) between consecutive positions, without immediate backtracking. We sweep k=2 . . . 8 and report recall against the exact top-n result and throughput (Figure 8; 95% CI over 90 instances, 3 seeds × 30).

PostgreSQL hands the whole predicate-filtered k-way self-join to the SQL planner: it uses the materialized edge relation, but produces a combinatorial intermediate. Milvus can apply per-position scalar filters, but cannot execute the cross-position join J or rank the paths. It therefore retrieves the per-position top-L nearest neighbors, applies P, and reranks the surviving length-k walks in the application layer. Given the materialized SemJI edge set, DASE computes the query as a *minimum-cost bounded-length walk* on the join graph and uses a Viterbi-like dynamic program with beam width n (the number of returned walks). It retains the top-n partial paths per directed-edge state (remembering the preceding vertex to exclude immediate backtracking); its worst-case time is $O\big(kn \sum_v \deg(v)^2\big)$, or $O(kn|E|)$ for bounded-degree graphs.

Both baselines fail as k grows (Figure 8). PostgreSQL's k-way join explodes, collapsing from 70 to 0.55 QPS over k=2 . . . 4 and timing out for $k \ge 5$. Milvus's qualifying sets are rarely nearest neighbors, so its recall decays (L=16: 0.78 → 0.27) and its L^k enumeration turns the accurate settings infeasible (L=128 by k=6). DASE keeps full recall for every k while sustaining 5,540 – 1,321 QPS, up to four orders of magnitude above PostgreSQL and never infeasible. The gap *widens* with k: the baselines time out or blow up while DASE stays flat, confirming that join- and predicate-aware materialization pays off exactly in the multistep regime.

5.5 Filter Sets in the ANN Index

Table 5 evaluates predicate-aware HNSW search strategies. The baseline returns the exact top-k answer (recall 1.0), but at low throughput (0.23 QPS). Filter pushdown with a Bitmap consistently delivers better recall and higher QPS than Bloom filters. With Bitmaps, we further evaluate graph traversal methods. The 2-hop adaptive strategy yields the optimal trade-off: by dynamically exploring extended neighbors only when valid nodes are sparse, it achieves a roughly 40× speedup (10.1 QPS) over the baseline while maintaining a highly competitive recall of 0.903.

Table 3: SemBench cost, quality, and latency (notation: F = filter; L = limit-k; J = join; M = semantic map; C = semantic classify; R = semantic rank)

	LOTUS [38]			Palimpzest [31]			ThalamusDB [22]			BigQuery [15]			DASE			DASE + BigQuery		
	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.
(a) Movie Scenario										*DASE Preprocess*			$3·10^{-3}		27.6 s	$3·10^{-3}		27.6 s
Q1: F L	$0.09	**1.00**	33.1 s	$1·10^{-3}	**1.00**	3.8 s	$4·10^{-4}	0.95	4.2 s	$0.05	**1.00**	26.3 s	$2.4·10^{-6}	**1.00**	0.5 s	$1·10^{-4}	**1.00**	3.8 s
Q2: F L	$0.01	**1.00**	2.1 s	$0.01	**1.00**	29.7 s	$2·10^{-3}	0.92	1.9 s	$3·10^{-3}	**1.00**	9.5 s	$2.4·10^{-6}	**1.00**	0.5 s	$1·10^{-4}	**1.00**	4.2 s
Q3: F	$5·10^{-3}	0.64	2.1 s	$0.01	0.64	4.6 s	$2·10^{-3}	0.74	3.1 s	$3·10^{-3}	0.64	11.0 s	$3.6·10^{-6}	**0.82**	0.5 s	$7·10^{-4}	**0.82**	6.2 s
Q4: F	$5·10^{-3}	0.64	2.8 s	$0.02	0.74	4.4 s	$2·10^{-3}	0.74	3.8 s	$3·10^{-3}	0.64	11.4 s	$3.6·10^{-6}	**0.82**	0.5 s	$7·10^{-4}	**0.82**	5.3 s
Q5: J L	$2.38	0.59	536.5 s	$0.01	0.39	1.9 s	$1·10^{-3}	**1.00**	2.3 s	$1.01	0.89	54.5 s	$3.6·10^{-6}	**1.00**	0.5 s	$6·10^{-4}	0.82	3.9 s
Q6: J L	$1.81	0.67	432.4 s	$0.01	0.83	2.3 s	$9·10^{-4}	0.84	1.7 s	$1.00	0.69	54.5 s	$3.6·10^{-6}	**1.00**	0.5 s	$7·10^{-4}	0.94	3.8 s
Q7: J	$1.81	0.21	431.8 s	$7.72	0.68	1056.1 s	$0.15	0.57	636.9 s	$3.31	0.70	198.3 s	$3.6·10^{-6}	**0.73**	0.5 s	$9·10^{-4}	0.71	3.8 s
Q8: C	$4·10^{-3}	**0.93**	2.3 s	$0.02	0.86	4.3 s	$5·10^{-3}	0.83	6.8 s	$3·10^{-3}	0.76	10.9 s	$3.6·10^{-6}	0.89	0.5 s	$7·10^{-4}	0.89	4.2 s
Q9: R	$0.02	0.75	4.9 s	$0.05	**0.78**	5.7 s	**X**	**X**	**X**	$0.02	**0.78**	13.3 s	$4.8·10^{-6}	0.66	0.5 s	$6·10^{-3}	0.74	12.8 s
Q10: R	$0.13	0.40	30.9 s	$0.38	0.42	39.2 s	**X**	**X**	**X**	$0.13	**0.44**	32.1 s	$4.8·10^{-6}	0.43	0.5 s	$0.085	0.41	26.1 s
Avg	$0.626	0.68	147.9 s	$0.823	0.73	115.2 s	$0.02	0.82	82.6 s	$0.553	0.75	42.2 s	$3·10^{-4}	**0.83**	3.0 s	$9·10^{-3}	0.81	9.2 s
(b) E-Commerce Scenario										*DASE Preprocess*			$0.03		124.0 s	$0.03		124.0 s
Q1: F	$0.06	**1.00**	12.2 s	$0.08	**1.00**	12.2 s	$0.03	**1.00**	34.8 s	$0.04	0.59	21.2 s	$5·10^{-6}	0.94	0.7 s	$7·10^{-3}	**1.00**	8.0 s
Q2: F	$0.18	**0.87**	166.1 s	$0.38	0.83	57.5 s	$0.31	0.67	100.8 s	$3.96	0.21	55.7 s	$7·10^{-6}	0.47	0.7 s	$0.02	0.66	5.5 s
Q3: M	$0.07	0.97	17.1 s	$0.12	**0.98**	16.5 s	**X**	**X**	**X**	$0.12	0.97	21.2 s	**X**	**X**	**X**	$0.04	0.93	10.5 s
Q4: M	$0.14	0.45	156.7 s	$0.24	0.53	335.0 s	**X**	**X**	**X**	$0.37	**0.69**	31.0 s	**X**	**X**	**X**	$0.17	0.67	18.3 s
Q5: C	$0.04	**0.99**	7.4 s	$0.07	0.98	6.6 s	**X**	**X**	**X**	$0.17	0.98	25.7 s	$4·10^{-6}	0.96	0.7 s	$0.01	0.98	5.8 s
Q6: C	$0.12	0.89	114.8 s	$0.43	0.89	143.7 s	**X**	**X**	**X**	$0.35	0.88	34.9 s	$4·10^{-6}	0.47	0.7 s	$0.16	**0.95**	19.5 s
Q7: J	$1.33	0.75	199.4 s	$1.79	**0.92**	287.6 s	$0.08	0.51	97.7 s	$0.86	0.83	45.4 s	$1·10^{-9}	0.78	2·10^{-3} s	$0.09	0.81	7.4 s
Q8: J	$4·10^{-3}	**1.00**	4.1 s	$0.01	**1.00**	3.9 s	$0.30	0.00	713.0 s	$18.23	0.29	126.2 s	$1·10^{-9}	0.25	1·10^{-3} s	$0.10	**1.00**	3.4 s
Q9: J	$0.06	0.55	243.4 s	$0.10	0.49	44.9 s	$0.31	0.00	872.2 s	$0.21	0.58	48.6 s	$1·10^{-9}	0.41	1·10^{-3} s	$0.07	**0.83**	18.8 s
Q10: F J	$0.21	0.00	519.5 s	$1.02	0.06	1192.5 s	**X**	**X**	**X**	**X**	**X**	**X**	$1·10^{-5}	**X**	0.7 s	$0.65	**0.34**	90.1 s
Q11: F J	$0.27	**0.78**	158.7 s	$0.61	0.73	132.4 s	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**	$1.68	0.40	131.3 s
Q12: F M	$0.10	0.60	36.4 s	$0.14	0.00	31.9 s	**X**	**X**	**X**	$0.10	**0.97**	31.1 s	$6·10^{-6}	0.85	0.7 s	$0.08	0.95	32.1 s
Q13: F	$0.24	**0.74**	274.8 s	$0.44	**0.74**	238.3 s	**X**	**X**	**X**	$0.38	0.70	22.4 s	$2·10^{-5}	0.58	1.0 s	$0.13	0.72	11.6 s
Q14: F J R	$0.23	0.87	178.2 s	**X**	**X**	**X**	**X**	**X**	**X**	$4.26	0.37	73.6 s	$5·10^{-6}	0.00	0.7 s	$4.83	**1.00**	87.3 s
Avg	$0.22	0.74	149.2 s	$0.42	0.70	192.5 s	$0.21	0.43	363.7 s	$2.42	0.67	44.8 s	$3·10^{-3}	0.57	10.8 s	$0.54	**0.80**	38.2 s
(c) Wildlife Scenario										*DASE Preprocess*			$0.01		53.9 s	$0.01		53.9 s
Q1: F	$0.11	**0.79**	92.4 s	$0.13	**0.79**	32.8 s	$0.11	**0.79**	19.6 s	$0.11	**0.79**	32.0 s	$1·10^{-9}	**0.79**	5·10^{-4} s	$0.02	0.78	10.1 s
Q2: F	**X**	**X**	**X**	$0.01	0.17	2.8 s	$0.01	0.14	4.5 s	$0.01	0.19	9.4 s	$1·10^{-9}	**0.83**	5·10^{-4} s	$3·10^{-3}	0.50	6.2 s
Q3: F L	$0.11	**1.00**	99.4 s	$0.13	0.00	22.7 s	$0.03	0.00	4.3 s	$0.11	0.00	25.7 s	$1·10^{-9}	0.00	8·10^{-4} s	$0.02	**1.00**	9.0 s
Q4: F L	**X**	**X**	**X**	$0.01	0.00	2.7 s	$1·10^{-3}	0.00	1.3 s	$0.01	**1.00**	9.4 s	$1·10^{-9}	**1.00**	8·10^{-4} s	$3·10^{-3}	**1.00**	6.1 s
Q5: F	**X**	**X**	**X**	$0.13	0.75	13.5 s	$0.01	0.75	2.3 s	$0.12	0.75	19.2 s	$1·10^{-9}	**1.00**	1·10^{-3} s	$0.02	0.85	7.0 s
Q6: F	**X**	**X**	**X**	$0.13	0.00	19.3 s	$0.06	**0.50**	13.2 s	$0.12	0.20	24.3 s	$1·10^{-9}	0.00	1·10^{-3} s	$0.06	0.40	13.9 s
Q7: F	$0.23	**1.00**	188.3 s	$0.13	**1.00**	43.9 s	$0.20	**1.00**	28.8 s	$0.22	**1.00**	24.6 s	$1·10^{-9}	0.00	1·10^{-3} s	$0.14	**1.00**	19.9 s
Q8: F	**X**	**X**	**X**	$0.13	**0.75**	34.9 s	$0.12	**0.75**	23.8 s	$0.23	**0.75**	35.5 s	$1·10^{-9}	0.00	2·10^{-3} s	$0.16	0.74	24.8 s
Q9: F	**X**	**X**	**X**	$0.13	0.57	19.1 s	$0.08	**0.67**	17.2 s	$0.12	0.59	37.6 s	$1·10^{-9}	0.00	1·10^{-3} s	$0.06	0.40	19.5 s
Q10: F L	$0.11	**1.00**	87.8 s	$0.13	0.00	19.6 s	$0.03	0.00	4.4 s	$0.11	0.00	40.3 s	$1·10^{-9}	**1.00**	8·10^{-4} s	$0.06	0.25	24.2 s
Avg	$0.14	**0.94**	117.0 s	$0.11	0.40	21.1 s	$0.07	0.46	11.9 s	$0.12	0.52	25.8 s	$9·10^{-4}	0.46	4.9 s	$0.05	0.69	17.7 s
(d) MMQA Scenario										*DASE Preprocess*			$0.01		50.1 s	$0.01		50.1 s
Q1: M	$0.02	**1.00**	7.4 s	$3·10^{-3}	**1.00**	4.5 s	**X**	**X**	**X**	$0.01	**1.00**	14.1 s	**X**	**X**	**X**	$0.01	**1.00**	14.1 s
Q2a: J	$0.89	0.83	169.6 s	$1.04	**1.00**	135.9 s	**X**	**X**	**X**	$0.08	0.00	53.8 s	$1·10^{-5}	0.08	1.0 s	$0.04	0.57	28.6 s
Q2b: J	$0.89	0.83	166.8 s	$1.04	**1.00**	133.8 s	**X**	**X**	**X**	$0.12	0.00	38.4 s	$1·10^{-5}	0.08	1.0 s	$0.06	0.57	21.4 s
Q3a: F	$0.01	**0.83**	4.2 s	$0.02	0.80	4.5 s	$0.01	0.75	11.0 s	$0.01	0.72	19.6 s	$5·10^{-6}	0.38	0.7 s	$9·10^{-3}	0.72	20.5 s
Q3f: F	$0.01	**1.00**	4.2 s	$0.02	**1.00**	8.0 s	$0.01	**1.00**	7.0 s	$0.01	0.67	22.3 s	$5·10^{-6}	0.09	0.7 s	$0.01	0.67	22.3 s
Q4: M	**X**	**X**	**X**	$5·10^{-3}	0.54	1.2 s	**X**	**X**	**X**	$2·10^{-3}	0.60	9.7 s	**X**	**X**	**X**	$2·10^{-3}	**0.61**	9.5 s
Q5: M	$1·10^{-3}	**1.00**	0.48 s	$1·10^{-3}	**1.00**	0.50 s	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**
Q6a: F	$0.01	**1.00**	4.4 s	$0.02	**1.00**	4.0 s	$2·10^{-3}	0.33	5.6 s	$4·10^{-3}	0.03	18.9 s	$5·10^{-6}	0.04	0.7 s	$3·10^{-3}	**1.00**	15.8 s
Q6b: F	$0.01	**1.00**	3.8 s	$0.02	**1.00**	4.1 s	$2·10^{-3}	**1.00**	5.5 s	$4·10^{-3}	0.04	17.3 s	$5·10^{-6}	0.01	0.7 s	$4·10^{-3}	0.87	20.4 s
Q6c: F	$0.01	**1.00**	4.6 s	$0.02	**1.00**	4.7 s	$2·10^{-3}	0.53	13.4 s	$4·10^{-3}	0.13	17.4 s	$5·10^{-6}	0.10	0.7 s	$4·10^{-3}	0.66	19.1 s
Q7: J	$13.61	0.32	2311.4 s	$15.65	0.31	2101.6 s	**X**	**X**	**X**	$1.18	0.00	91.7 s	$1·10^{-5}	0.10	0.7 s	$0.04	**1.00**	6.9 s
Avg	$1.55	**0.88**	267.7 s	$1.62	0.87	218.4 s	$5·10^{-3}	0.72	8.5 s	$0.14	0.31	30.3 s	$1·10^{-3}	0.11	6.3 s	$0.02	0.76	20.8 s
(e) Cars Scenario										*DASE Preprocess*			$2.50		1300 s	$2.50		1300 s
Q1: F	$1.74	**0.90**	550.0 s	$2.44	0.69	465.6 s	$1.37	0.81	829.7 s	$1.44	0.71	61.7 s	$5·10^{-6}	0.66	0.9 s	$0.31	0.78	22.0 s
Q2: F	**X**	**X**	**X**	$4·10^{-3}	0.00	4.1 s	$0.01	0.09	14.1 s	$0.01	0.08	14.1 s	$5·10^{-6}	0.00	0.7 s	$1·10^{-3}	**0.16**	5.1 s
Q3: F L	$0.60	0.90	456.2 s	$0.01	0.92	6.1 s	**X**	**X**	**X**	$1.66	**1.00**	36.4 s	$5·10^{-6}	**1.00**	0.7 s	$2·10^{-3}	**1.00**	4.3 s
Q4: F	$1.71	0.99	822.0 s	$2.41	0.99	443.6 s	$1.34	**1.00**	768.8 s	$1.41	0.99	68.7 s	$5·10^{-6}	0.99	0.9 s	$0.31	0.99	23.4 s
Q5: F	**X**	**X**	**X**	$0.01	**1.00**	6.3 s	$1.61	**1.00**	483.7 s	$1.47	**1.00**	58.9 s	$1·10^{-5}	0.50	1.1 s	$7·10^{-3}	**1.00**	5.1 s
Q6: F J	**X**	**X**	**X**	$2.51	0.96	427.3 s	$1.96	**0.97**	775.4 s	$2.00	0.96	44.3 s	$1·10^{-5}	0.88	1.3 s	$0.89	0.96	41.2 s
Q7: F	**X**	**X**	**X**	$4.47	0.56	882.7 s	$3.06	**0.58**	2146.2 s	$3.17	0.45	86.0 s	$1·10^{-5}	0.47	1.3 s	$1.55	0.52	131.5 s
Q8: F L	$1.78	**0.45**	1349.8 s	$2.02	0.29	268.7 s	$0.21	0.20	68.0 s	$1.69	0.24	38.2 s	$5·10^{-6}	0.15	0.7 s	$0.04	0.39	6.0 s
Q9: F	**X**	**X**	**X**	$2.05	0.00	308.1 s	$1.61	**1.00**	335.8 s	$0.03	0.00	13.6 s	$5·10^{-6}	**1.00**	0.7 s	$0.03	0.00	16.4 s
Q10: C	$3.09	0.41	618.1 s	$4.69	0.51	594.9 s	**X**	**X**	**X**	$2.70	**0.57**	62.0 s	$2·10^{-5}	0.45	1.8 s	$0.86	0.55	27.6 s
Avg	$1.78	**0.73**	759.2 s	$2.06	0.59	340.7 s	$1.40	0.70	677.7 s	$1.56	0.60	48.4 s	$0.23	0.61	119.1 s	$0.59	0.63	143.9 s

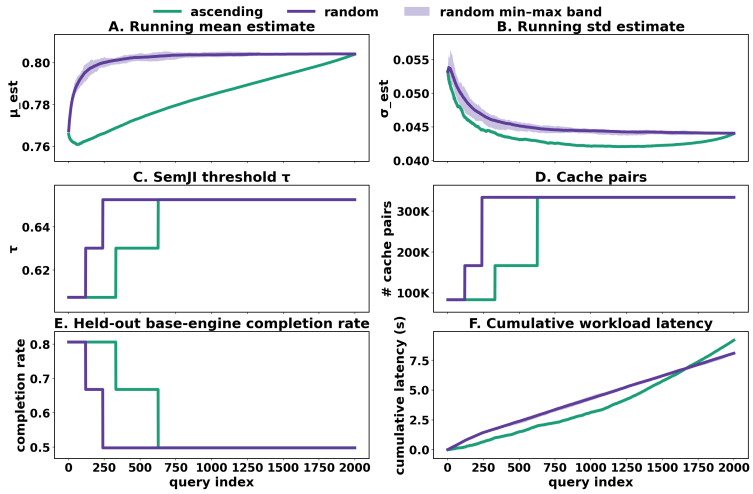


Figure 7: SemJI evolution test.

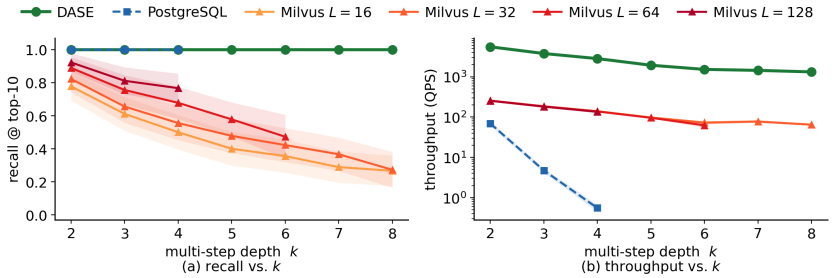


Figure 8: Number of Join Steps vs recall in NFCorpus

Table 4: SemJI Construction.

Method	Recall	Time
Exact Construction	1.00	~81.2 h
Iterative	0.98	14.0 m
Batched	0.98	3.0 m
Fully Vectorized	0.98	1.6 m

Table 5: Filtered search results.

Method	Setting	Recall	QPS
baseline	-	1.0	0.23
Bitmap	2-hop on	0.905	9.6
	2-hop off	0.884	10.5
	adaptive	0.903	10.1
Bloom	4 bits/element	0.817	9.1
	12 b/element	0.872	8.8
	20 b/element	0.883	8.2

Table 6: Component ablation on IMDB. Each mechanism is disabled in isolation; we report recall / throughput.

Mechanism	Variant	Recall	QPS
SemJI (W5)	DASE (with SemJI)	0.95	92.0
	− SemJI (on-the-fly join)	0.71	5.3
Stream selection (W3)	DASE (adaptive)	0.98	57.8
	− adaptive (round-robin)	0.98	29.8

5.6 DASE Ablation Study

We isolate DASE's central algorithmic contributions by disabling one at a time on the workload that exercises it in isolation, holding everything else fixed (IMDB, 100 queries, warm cache).

SemJI join materialization (Table 6, top). On the pure semantic-join workload W5 we keep DASE's seed streamer unchanged and switch *only* the partner check: the full engine probes the materialized SemJI index, while the ablated variant resolves each candidate's partner on the fly with an ANN query against the partner table. Removing SemJI is strictly worse on *both* axes — recall drops from 0.95 to 0.71 and throughput falls 17× (92.0 → 5.3 QPS) — because without the precomputed join the partner test must either approximate (missing qualifying partners, hence the recall loss) or run exact (far slower still). Materializing the join is what makes the join-bearing workloads both accurate and fast.

Distribution-aware stream selection (Table 6, bottom). On the pure multi-scoring workload W3 we replace DASE's threshold-algorithm stream selector — which extends the stream with the largest expected gap reduction $G_j = w_j/\mathrm{PDF}(x_j)$ — with vanilla round-robin extension. At the exact operating point (ε=0, recall 0.98) adaptive selection delivers 1.9× the throughput (57.8 vs 29.8 QPS): choosing the right stream closes the termination gap in far fewer extension rounds. The gain is largest exactly here, in the high-recall regime where many rounds are needed, and concentrates on queries whose signals have heterogeneous score distributions.

5.7 Non-Gaussian Embedding Distributions

The *materialization radius* τ_{cap} retains every endpoint pair within $\mathrm{dist}(a, b) \le \tau_{cap}$; SemJI is the union of these per-endpoint neighborhoods. In a query with multiple similarity and Boolean predicates, due to anisotropy, a spherical raw-distance cutoff for SemJI may be difficult to calibrate to fit the set of near-neighbors.

To understand whether this occurs in practice, we examined the distribution of embedding distances for the IMDB and MOLECULE W7/W8 join endpoints. Indeed, out of 1,536 dimensions in the embedding, their covariance participation ratios are 75–93 (IMDB) and 112–166 (MOLECULE), suggesting variance carried along only 5–11% of the embedding dimensions.

We next compared the methods described in Section 3.3, to see how well they can adjust for anisotropy and provide a better SemJI distance measure. For each corpus, we ran 200 queries over two independent 3K-by-3K reservoir samples (400 query–sample units per arm). We also computed an oracle set of actual top answers.

During execution, we apply predicates before ranking and macro-average recall@20/precision@20 against the top-20 oracle. Each arm calibrates τ_{cap} in its own retrieval space; PCA uses 0.1 shrinkage and ABTT removes one leading component, both fit on each sample's 6K endpoints. Results are as follows.

Table 7 shows that the raw workload cutoff strategy loses substantial coverage, especially on MOLECULE. **ABTT** has the highest within-arm coverage (0.991/1.000 recall/precision on IMDB and 0.969/0.969 on MOLECULE). At the same materialized-pair fraction,

Table 7: Predicate-filtered SemJI quality. Each Rec./ Prec. entry is recall@20 / precision@20. % Pairs represents the fraction of possible pairs materialized in SemJI.

Arm	IMDB Rec./ Prec.; % Pairs	MOLECULE Rec./ Prec.; % Pairs
Raw workload cutoff	0.795 / 0.842 ; 0.60%	0.506 / 0.700 ; 0.03%
Raw $\mu - 3\sigma$	0.920 / 0.929 ; 0.42%	0.867 / 0.867; 0.62%
PCA whitening	0.910 / 0.920 ; 0.44%	0.868 / 0.868 ; 0.47%
ABTT	**0.991 / 1.000; 1.09%**	**0.969 / 0.969 ; 0.93%**
Raw radius control	0.954 / 0.962 ; 1.09%	0.881 / 0.881 ; 0.93%

the **raw radius control** has lower recall — demonstrating that transformed-space calibration therefore improves which pairs are retained, not merely their number. Finally, the SemJI remains much smaller than the Cartesian product (at most 1.09%).

6 Related Work

Semantic-operator systems. LOTUS [38] and Palimpzest [31] optimize declarative operations whose predicates, joins, or transformations may be LLM-based. ThalamusDB [22, 47] similarly applies approximate query processing to image and text data. SemBench [27] compares these systems and others on semantic-filter, semantic-map, and join tasks. DASE complements these systems: assuming that a semantic-query planner has supplied a retrieval plan and scoring function, it constructs and ranks a high-recall candidate set that can be fed into an LLM-based semantic judgment and filtering stage (as in our DASE + BigQuery configuration). Optimizations have been studied in an operator-specific way, including BARGAIN [64] for filtering and classification, and FDJ [65] for semantic joins. DASE instead supports prefiltering across multiple semantic operators and integrates joins with top-k retrieval under monotone multi-signal scoring functions.

Semantic query benchmarks. MMQA [57] evaluates LLMs on ranked multi-table retrieval, question answering, SQL generation, and identification of primary and foreign keys. STaRK [58] benchmarks entity retrieval under combined textual and relational requirements, including multi-hop relations. FDABench [52] evaluates analytical agents that integrate structured and unstructured sources, measuring answer quality, tool use, and end-to-end cost and latency. These benchmarks are based on natural-language requests. Our benchmark instead takes SQL queries and a multi-signal scoring function as given, and measures the recall–throughput trade-off for jointly scored evidence tuples. Its workloads combine structured constraints, exact joins, and thresholded embedding-similarity joins, with exhaustive top-k reference results defined by the fixed query and scoring semantics.

Record linking. Increasingly, record linking [12] tasks are being addressed using embeddings and LLMs [63]. SemJI can be viewed as a materialized snapshot of links above a threshold.

Hybrid DBMSs. Milvus [50] combines vector retrieval with scalar filtering, while filtered ANNs [18, 34, 39, 51] improve **point-query vector search** under structured predicates. These methods are useful building blocks for DASE's filtered score streams, but their access pattern matches one query vector against an indexed collection. DASE targets include M-to-N embedding-similarity joins between distinct relational rows, and exact joins that connect those rows into a pattern, with a monotone score that combines several lookup and join affinities. SemJI materializes a sparse subset of those cross-table pairs so that the resulting pattern can be evaluated with relational operations rather than repeated point lookups.

Nova [7] optimizes query-vector top-k retrieval with joins between embedding and metadata tables, while Presto Vector Search [59] maps batched per-query ANN search to relational operators that compose with filters, joins, and aggregations. Neither system provides dedicated execution support for joint top-k ranking of cross-table evidence tuples under a monotone score combining lookup affinities with cross-row embedding-similarity links, as required for multistep reasoning. DASE supports this joint ranking through SemJI access to thresholded similarity pairs, filtered score streams, and threshold-based aggregation.

Ranked join processing. Threshold-style top-k algorithms [14, 24, 48] combine sorted score streams with random access and stopping bounds. DASE adopts this execution principle: an embedding-similarity-join score belongs to a pair of rows that must be connected through the query pattern before it contributes to an output tuple. The materialized join relation, filtered score streams, and Multi-Scoring Aggregator provide the access paths and bound-aware coordination needed for this setting.

Embedding geometry. Embedding whitening and anisotropy correction are well studied for NLP sentence and word representations [2, 13, 36, 46]. We use these techniques in a different role: when anisotropic or shared-context endpoint embeddings make a distribution-derived materialization radius unreliable, DASE separately calibrates a transformed embedding space and constructs the corresponding embedding-similarity join index there.

7 Conclusions

Multistep reasoning queries frequently arise in agentic data engineering tasks. We introduced SemJI, which facilitates efficient candidate exploration in top-k multistep reasoning queries, along with ANN extensions to enable batched requests. We developed DASE, a query processing strategy that jointly optimizes attribute predicates and vector similarity scoring across joins. Our extensive evaluation, over existing semantic benchmarks and a scientific-discovery workload that captures the structure of multistep reasoning queries, demonstrates the retrieval benefits and approximation trade-offs of SemJI. We also evaluated a robustness treatment for anisotropic endpoint embeddings, where a raw distribution-derived SemJI materialization radius can miss useful links.

We hope that our formalization, methods, and benchmarks serve as a foundation for continued progress on query systems that bridge structured and unstructured data at scale. In ongoing work, we are focused on developing a high-level planner that uses costs to break agentic plans into multistep reasoning stages.

AI Acknowledgment

The authors used Anthropic Claude Opus 4.6 and Google Gemini 3.5 to help edit portions of the manuscript for length and clarity. The core ideas, algorithmic insights, and conclusions are our own. Claude Code 4.5 was used to aid with implementation. Extensive specification and testing were done to ensure correctness.

References

[1] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A. Shah, Ben Sowell, Dan G. Tecuci, Vinayak Thapliyal, and Matt Welsh. 2024. The Design of an LLM-powered Unstructured Analytics System. *ArXiv* abs/2409.00847 (2024). https://api.semanticscholar.org/CorpusID:272368251

[2] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A simple but tough-to-beat baseline for sentence embeddings. In *International conference on learning representations*.

[3] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, et al. 2026. Synthesizing scientific literature with retrieval-augmented language models. *Nature* 650, 8103 (2026), 857–863.

[4] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *CACM* 13, 7 (July 1970), 422–426.

[5] Vera Boteva, Demian Gholipour, Artem Sokolov, and Stefan Riezler. 2016. A full-text learning to rank dataset for medical information retrieval. In *European Conference on Information Retrieval*. Springer, 716–722.

[6] Huiping Cao, Yan Qi, K Selçuk Candan, and Maria Luisa Sapino. 2010. Feedback-driven result ranking and query refinement for exploring semi-structured data collections. In *EDBT*. 3–14.

[7] Dechuang Chen, Bing Chen, Jianwei Lu, Zhiwei Wu, Zeyu Yang, Qiang Gu, Fangyuan Zhang, Sibo Wang, Zeyuan Yu, Caihua Yin, Wenchao Zhou, and Feifei Li. 2026. Nova: A Multi-Purpose Vector Engine for Low-Latency, Multi-Tenant, and Cross-Table Hybrid Retrieval. *Proceedings of the VLDB Endowment* 19, 12 (2026), 4453–4465. doi:10.14778/3827998.3828045

[8] Mingju Chen, Guibin Zhang, Heng Chang, Yuchen Guo, and Shiji Zhou. 2026. A-MapReduce: Executing Wide Search via Agentic MapReduce. *arXiv preprint arXiv:2602.01331* (2026).

[9] Xu Chen, Zheng Qin, Yongfeng Zhang, and Tao Xu. 2016. Learning to rank features for recommendation over multiple categories. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. 305–314.

[10] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. 2020. ChemBERTa: Large-Scale Self-Supervised Pretraining for Molecular Property Prediction. doi:10.48550/arXiv.2010.09885 arXiv:2010.09885.

[11] Peter Christen. 2011. A survey of indexing techniques for scalable record linkage and deduplication. *IEEE transactions on knowledge and data engineering* 24, 9 (2011), 1537–1555.

[12] Peter Christen. 2012. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer.

[13] Kawin Ethayarajh. 2019. How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 55–65.

[14] Ronald Fagin, Amnon Lotem, and Moni Naor. 2003. Optimal aggregation algorithms for middleware. *Journal of computer and system sciences* 66, 4 (2003), 614–656.

[15] Sérgio Fernandes and Jorge Bernardino. 2015. What is bigquery?. In *Proceedings of the 19th International Database Engineering & Applications Symposium*. 202–203.

[16] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *ArXiv* abs/2312.10997 (2023). https://api.semanticscholar.org/CorpusID:266359151

[17] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. 2026. A multi-agent system for automating scientific discovery. *Nature* (2026), 1–3.

[18] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, and others. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.

[19] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. 2026. Accelerating scientific discovery with Co-Scientist. *Nature* (2026), 1–3.

[20] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (Sept. 2025), 633–638. doi:10.1038/s41586-025-09422-z

[21] Ziyang Huang, Haolin Ren, Xiaowei Yuan, Jiawei Wang, Zhongtao Jiang, Kun Xu, Shizhu He, Jun Zhao, and Kang Liu. 2026. WideSeek: Advancing Wide Research via Multi-Agent Scaling. *arXiv preprint arXiv:2602.02636* (2026).

[22] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proceedings of the ACM on Management of Data* 2, 3 (May 2024), 1–26. doi:10.1145/3654989

[23] Alfons Kemper and Guido Moerkotte. 1990. Access Support in Object Bases. In *SIGMOD*. 364–374.

[24] Benny Kimelfeld and Yehoshua Sagiv. 2006. Finding and approximating top-k answers in keyword proximity search. In *PODS*. 173–182.

[25] Tian Lan, Felix Henry, Bin Zhu, Qianghuai Jia, Junyang Ren, Qihang Pu, Haijun Li, Longyue Wang, Zhao Xu, and Weihua Luo. 2026. Table-as-Search: Formulate Long-Horizon Agentic Information Seeking as Table Completion. *arXiv preprint arXiv:2602.06724* (2026).

[26] Tian Lan, Bin Zhu, Qianghuai Jia, Junyang Ren, Haijun Li, Longyue Wang, Zhao Xu, Weihua Luo, and Kaifu Zhang. 2025. Deepwidesearch: Benchmarking depth and width in agentic information seeking. *arXiv preprint arXiv:2510.20168* (2025).

[27] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. doi:10.48550/arXiv.2511.01716 arXiv:2511.01716.

[28] Chengkai Li, Kevin Chen-Chuan Chang, Ihab F. Ilyas, and Sumin Song. 2005. RankSQL: Query Algebra and Optimization for Relational Top-k Queries. In *SIGMOD*. 131–142.

[29] Hang Li. 2011. *Learning to Rank for Information Retrieval and Natural Language Processing*. Morgan Claypool.

[30] Jisen Li, Bingxuan Li, Nanyi Jiang, Xuying Ning, Xiyao Wang, Yifan Shen, Heng Wang, Yuqing Jian, Xiaoxia Wu, Ben Athiwaratkun, et al. 2026. Multi-Turn Agentic Scientific Literature Search via Workflow Induction. *arXiv preprint arXiv:2607.00597* (2026).

[31] Chunwei Liu, Matthew Russo, Michael J Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael J Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing.. In *CIDR*.

[32] Shu Liu, Soujanya Ponnapalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, et al. 2025. Supporting our ai overlords: Redesigning data systems to be agent-first. *arXiv preprint arXiv:2509.00997* (2025).

[33] Duo Lu, Helena Caminal, Manos Chatzakis, Yannis Papakonstantinou, Yannis Chronis, Vaibhav Jain, and Fatma Özcan. 2026. An in-depth study of filter-agnostic vector search on a postgresql database system:[experiments & analysis]. *Proceedings of the ACM on Management of Data* 4, 3 (SIGMOD (2026), 1–26.

[34] Jiarui Luo, Chaoji Zuo, and Dong Deng. 2026. CGIF: Combining Proximity Graphs and Inverted Files for Efficient Filtered Vector Search over Arbitrary Predicates. *Proceedings of the VLDB Endowment* 19, 11 (2026), 3663–3675. doi:10.14778/3836663.3836716

[35] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[36] Jiaqi Mu and Pramod Viswanath. 2018. All-but-the-Top: Simple and Effective Postprocessing for Word Representations. In *International Conference on Learning Representations (ICLR)*.

[37] Rodrigo Nogueira and Kyunghyun Cho. 2020. Passage Re-ranking with BERT. doi:10.48550/arXiv.1901.04085 arXiv:1901.04085.

[38] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184.

[39] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[40] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. doi:10.48550/arXiv.2408.08921 arXiv:2408.08921.

[41] Pinecone Systems. 2026. Pinecone: The Vector Database for AI Applications. https://www.pinecone.io/. Accessed: 2026-04-15.

[42] David Rogers and Mathew Hahn. 2010. Extended-Connectivity Fingerprints. *Journal of Chemical Information and Modeling* 50, 5 (May 2010), 742–754. doi:10.1021/ci100050t

[43] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *arXiv preprint arXiv:2505.14661* (2025).

[44] Ahmmad OM Saleh, Gokhan Tur, and Yucel Saygin. 2024. SG-RAG: Multi-hop question answering with large language models through knowledge graphs. In *Proceedings of the 7th International Conference on Natural Language and Speech Processing (ICNLSP 2024)*. 439–448.

[45] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3035–3048.

[46] Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. 2021. Whitening Sentence Representations for Better Semantics and Faster Retrieval. *arXiv preprint arXiv:2103.15316* (2021). doi:10.48550/arXiv.2103.15316 arXiv:2103.15316.

[47] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. doi:10.48550/arXiv.2510.08489 arXiv:2510.08489.

[48] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2024. Ranked Enumeration for Database Queries. *ACM SIGMOD Record* 53, 3 (Nov. 2024), 6–19. doi:10.1145/3703922.3703924

[49] Roman Vershynin. 2018. *High-dimensional probability: An introduction with applications in data science*. Vol. 47. Cambridge university press.

[50] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 2614–2627. doi:10.1145/3448016.3457550

[51] Ziqi Wang, Jingzhe Zhang, Shuo Shen, and Wei Hu. 2026. Fast Label-Filtering Approximate Nearest Neighbor Search via Progressive Label Set Stratification. arXiv:2609.15058. https://arxiv.org/abs/2609.15058 Accepted at ACM SIGMOD 2027.

[52] Ziting Wang, Shize Zhang, Haitao Yuan, Jinwei Zhu, Wei Dong, and Gao Cong. 2026. FDABench: A Benchmark for Data Agents on Analytical Queries over Heterogeneous Data. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. ACM, New York, NY, USA, 9985–9996. doi:10.1145/3770855.3817454

[53] Weaviate. 2026. Weaviate: An Open-Source Cloud-Native Vector Database. https://github.com/weaviate/weaviate. Accessed: 2026-04-15.

[54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[55] David Weininger. 1988. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of chemical information and computer sciences* 28, 1 (1988), 31–36.

[56] Ryan Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, Wenhao Huang, Yang Wang, and Ke Wang. 2025. WideSearch: Benchmarking Agentic Broad Info-Seeking. arXiv:2508.07999 [cs.CL] https://arxiv.org/abs/2508.07999

[57] Jian Wu, Linyi Yang, Dongyuan Li, Yuliang Ji, Manabu Okumura, and Yue Zhang. 2025. MMQA: Evaluating LLMs with multi-table multi-hop complex questions. In *International Conference on Learning Representations*, Vol. 2025. 48626–48643.

[58] Shirley Wu, Shiyu Zhao, Michihiro Yasunaga, Kexin Huang, Kaidi Cao, Qian Huang, Vassilis N. Ioannidis, Karthik Subbian, James Zou, and Jure Leskovec. 2024. STaRK: Benchmarking LLM Retrieval on Textual and Relational Knowledge Bases. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 127129–127153. doi:10.52202/079017-4037

[59] Zhichen Xu, Ge Gao, Ke Wang, Aakash Deep, Junjie Qi, Jort Gemmeke, Amit Dutta, Xiaoxuan Meng, Trang Nguyen, Jiayin Yan, Xiao Du, Vivek Gaur, Kaushik Ravichandran, and Vishal Gandhi. 2026. SQL-Native Vector Search at Billion Scale in Presto. *Proceedings of the VLDB Endowment* 19, 12 (2026), 4182–4194. doi:10.14778/3827998.3828025

[60] Zelai Xu, Zhexuan Xu, Ruize Zhang, Chunyang Zhu, Shi Yu, Weilin Liu, Quanlu Zhang, Wenbo Ding, Chao Yu, and Yu Wang. 2026. WideSeek-R1: Exploring Width Scaling for Broad Information Seeking via Multi-Agent Reinforcement Learning. arXiv:2602.04634 [cs.AI] https://arxiv.org/abs/2602.04634

[61] Zhepeng Yan, Nan Zheng, Zachary G Ives, Partha Pratim Talukdar, and Cong Yu. 2013. Actively soliciting feedback for query answers in keyword search-based data integration. *Proceedings of the VLDB Endowment* 6, 3 (2013), 205–216.

[62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[63] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. 2025. AvengER: Ensembling and Fine-Tuning LLMs for SELECT Prompts in Entity Resolution. In *European Semantic Web Conference*. Springer, 301–320.

[64] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2025. Cut costs, not accuracy: Llm-powered data processing with guarantees. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[65] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2026. Featurized-Decomposition Join: Low-Cost Semantic Joins with Guarantees. *Proc. VLDB Endow.* 19, 11 (Sept. 2026), 3048–3061. doi:10.14778/3836663.3836672

[66] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 414–431.

Appendix A Benchmark Dataset

A.1 IMDB Benchmark

A.1.1 Dataset Description. The IMDB dataset is designed to simulate a movie discovery and recommendation environment. It allows for querying films based on a combination of categorical metadata (e.g., genres, ratings, release years) and rich textual profiles. To set up a scenario facilitating cross-era movie discovery (e.g., finding modern successors to classic cinema), we partition the data into two relational tables, T_1 and T_2, representing different time periods. Each record encapsulates a comprehensive movie profile, including a synthesized crew description and high-dimensional embeddings for its plot and title.

A.1.2 Data Source and Construction. To construct this benchmark, we integrated official metadata with unstructured content:

- **Data Sourcing:** We extracted structured attributes (title, rating, crew identifiers) from the official IMDB non-commercial datasets. To ensure a high-quality corpus, we limited the scope to non-adult feature films released between 1950 and 2020. This was augmented with detailed plot descriptions sourced from the `jquigl/imdb-genres` collection.
- **Profile Construction:** We synthesized an `actor_director` textual field for each movie by joining the crew identifiers with the name basics table, resolving top-billed cast members and directors into human-readable strings. The final consolidated table, consisting of 85,615 cleaned records, was merged on the composite key of movie title and release year.
- **Vectorization / Partition:** We generated 1536-dimensional embeddings for the `title`, `plot`, and `actor_director` fields using the Gemini-embedding-001 model. Finally, the dataset was split by the median release year (2007) into table `imdb_t1` (42,378 records, pre-2007) and table `imdb_t2` (43,237 records, 2007–2020) to provide physical targets for relational semantic joins.

A.1.3 Workload Construction. To systematically evaluate the execution strategies, we generate a synthetic workload suite that instantiates the three core dimensions of our multistep reasoning query model: structured predicates ($\mathcal{P}$), multi-vector scoring ($\mathcal{S}$), and semantic join conditions ($\mathcal{J}$).

Predicates and Scoring ($\mathcal{P}$ and $\mathcal{S}$). We instantiate hard logical constraints ($\mathcal{P}$) using the exact relational metadata, such as bounding the release years or applying exact-match filters on categorical genres. The multi-dimensional ranking objective ($\mathcal{S}$) is formulated by aggregating the semantic similarities between user-provided query anchors and the movies' high-dimensional `plot` and `title` embeddings.

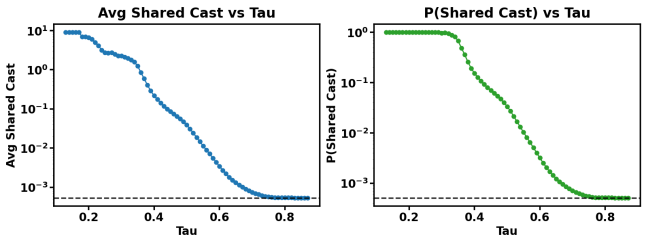


Figure 9: Validation of the semantic join proxy: shared cast metrics versus embedding distance τ. Dashed lines indicate global baselines.

Semantic Joins ($\mathcal{J}$). To evaluate inter-entity synthesis (workloads W5–W8), we simulate a cross-era discovery scenario that links classic films in `imdb_t1` with their modern counterparts in `imdb_t2`. This linkage is fundamentally driven by a semantic join over the `actor_director` embeddings.

To validate that the geometric distance (τ) in this specific embedding space serves as a robust proxy for real-world semantic relatedness, we profiled the pairwise distances across the dataset. As illustrated in Figure 9, we measured both the average number of shared cast members (left) and the probability of two movies sharing at least one cast member (right) as a function of the distance threshold τ. Note that the y-axes are scaled logarithmically, and the horizontal dashed lines denote the global expected baselines across the unconstrained Cartesian product.

The empirical results exhibit a pronounced correlation: at tight distance thresholds (small τ), both the expected density and the probability of shared cast members increase exponentially, vastly outperforming the global average. This observation perfectly aligns with our geometric premise that meaningful semantic affinities manifest strictly as extreme tail events. Consequently, this justifies our methodology of utilizing the `actor_director` embedding distance as the underlying relational proxy to construct and evaluate ground-truth semantic join pairs.

W1: Find movies by a plot description (Semantic Retrieval). To establish a performance baseline, we synthesize a query set of 100 seed phrases simulating authentic user search behaviors. We generate these search intents using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] short phrases that someone might type when searching for movies. Examples: 'thriller with twist', 'romantic comedy in Paris'. Output one phrase per line, no numbering, no other text."* Each generated phrase is then vectorized into 1536-dimensional embeddings using the Gemini-embedding-001 model. To ensure uniform coverage across the partitioned corpus, the generator randomly assigns each query to one of the four possible target spaces: either the `plot` or `title` embeddings within `imdb_t1` or `imdb_t2`. Each query is parameterized with $K = 20$. This workload represents the simplest form of single-signal semantic retrieval, providing the ground-truth baseline for the system's raw vector search throughput without structural constraints.

```
-- Searching for "a thriller with unexpected twist and
    mystery"
SELECT tconst, title, plot
FROM imdb_t1
ORDER BY plot_emb <-> '[0.0336, -0.0163, ...]'
LIMIT 20;
```

Listing 2: Example W1 Query

W2: Find movies by a plot description, within genre/era (Filtered Retrieval). Building upon W1, workload W2 introduces exact structured predicates ($\mathcal{P}$) to evaluate filtered semantic search. To construct this workload, we keep the same query-text prompt

setting as W1, generate 100 intents, and vectorize each into a 1536-dimensional embedding. We then augment each query with 1 to 3 relational filters applied strictly to target-table metadata. The predicate space is bounded by three explicit rules: (1) temporal constraints applying $\geq$ or $\leq$ operators on `year` within $[1980, 2020]$; (2) quality thresholds applying the $\geq$ operator on `rating` within $[1.0, 10.0]$; and (3) categorical constraints applying the IN operator over random subsets of 12 predefined cinematic `genres`. Paired with a uniformly selected target semantic space (either `plot_emb` or `title_emb` within `imdb_t1` or `imdb_t2`) and parameterized with $K = 20$, this workload benchmarks hybrid execution with relational pruning plus dense vector scans.

```
-- Q: "a thriller with unexpected twist and mystery"
-- Filtered by specific categorical genres
SELECT tconst, title, genre, plot
FROM imdb_t1
WHERE genre IN ('Action', 'Fantasy', 'Thriller')
ORDER BY plot_emb <-> '[-0.007, -0.028, ...]'
LIMIT 20;
```

Listing 3: Example W2 Query

W3: Find movies matching a multi-aspect semantic profile (Multi-scoring retrieval). To evaluate multi-signal aggregation (Multi-$\mathcal{S}$) over a single entity without structural predicates ($\mathcal{P} = \emptyset$), workload W3 synthesizes user intents spanning multiple semantic dimensions. We generate 100 queries uniformly targeting either `imdb_t1` or `imdb_t2`. Query text generation uses a 3-slot prompt template (plot description | title style | actor/director vibe), then maps each slot to `plot_emb`, `title_emb`, and `actor_director_emb`, respectively. Each component phrase is embedded with Gemini-embedding-001 (1536-d). Ranking uses a late-binding weighted sum over these L_2 vector distances, with per-query weights normalized to 1.0 and $K = 20$.

```
-- Q: Find movies in imdb_t1 matching a multi-aspect
     profile:
-- famous thriller actors AND unexpected twist plot motif
SELECT tconst, title, year
FROM imdb_t1
ORDER BY
  0.60 * (actor_director_emb <-> '[-0.033, -0.031, ...]')
  -- demographic cast intent mapping (L2 distance)
+ 0.40 * (plot_emb <-> '[-0.007, -0.028, ...]')
  -- narrative plot intent mapping (L2 distance)
LIMIT 20;
```

Listing 4: Example W3 Query

W4: Find movies matching a multi-aspect semantic profile within strict relational filters (Filtered multi-scoring). Building upon W3, workload W4 introduces exact structural predicates ($\mathcal{P}$) to evaluate filtered multi-signal aggregation (Multi-$\mathcal{S}$) over a single entity without cross-table joins ($\mathcal{J} = \emptyset$). We synthesize 1000 queries uniformly targeting either `imdb_t1` or `imdb_t2`. Query text generation follows the same three-slot semantics as W3, using a tab-separated triple format to improve parsing robustness at larger scale. The three text slots are mapped to `plot_emb`, `title_emb`, and `actor_director_emb`, and each is embedded with Gemini-embedding-001 (1536-d).

To instantiate the structured predicates, each query is dynamically augmented with 1 to 2 relational filter groups applied strictly to the target table's metadata. The predicate generation space is strictly bounded by three explicit logical rules: (1) temporal constraints applying the $\geq$ operator on `year` sampled within $[1983, 2008]$, or applying both $\geq$ and $\leq$ operators bounded by intervals $[1983, 2003]$ and $[1995, 2018]$; (2) quality thresholds applying the $\geq$ operator on `rating` sampled within $[3.5, 7.5]$, or both $\geq$ and $\leq$ operators bounded by $[3.5, 6.5]$ and $[5.0, 9.0]$; and (3) categorical constraints applying the IN operator over random subsets of 1 to 4 predefined cinematic `genres` from a pool of 12 options. Logically, the query formulates a top-K ($K = 20$) retrieval where the final ranking is determined by a late-binding, weighted sum of the L_2 distances across 2 to 3 corresponding continuous vector spaces. Each component signal is assigned a randomized interpolation weight summing to 1.0. This workload benchmarks the capability of the database engine to efficiently execute hybrid queries combining diverse relational filters with dense multi-stream vector scans.

```
-- Q: Find movies in imdb_t2 matching a multi-aspect
     profile:
-- "enchanting romance film" (title), "magical romance
     movie stars" (cast),
-- and "romantic fantasy involving mythical realms" (plot
     )
SELECT tconst, title, year, genre
FROM imdb_t2
WHERE year >= 2005
  AND genre IN ('Adventure', 'Mystery')
ORDER BY
  0.440 * (plot_emb <-> '[-0.028, -0.012, ...]')
  -- narrative plot intent (L2 distance)
+ 0.426 * (actor_director_emb <-> '[-0.039, -0.034, ...]'
     )
  -- demographic cast intent (L2 distance)
+ 0.134 * (title_emb <-> '[-0.058, -0.016, ...]')
  -- stylistic title intent (L2 distance)
LIMIT 20;
```

Listing 5: Example W4 Query

W5: Pair up similar-cast films that fit a plot or title theme (Semantic join). Transitioning to Inter-Entity Synthesis ($\mathcal{J}$), workload W5 evaluates semantic joins without structural predicates ($\mathcal{P} = \emptyset$). At a high level, this query pattern represents a cross-era discovery task: it finds pairs of classic and modern movies that share a highly identical demographic cast or crew, and ranks these linked pairs globally based on how well one of the movies matches a specific narrative or stylistic search intent. To construct this workload, we synthesize 100 movie search intents using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] short phrases that someone might type when searching for movies. Examples: 'thriller with twist', 'romantic comedy in Paris'. Output one phrase per line, no numbering, no other text."* Each generated phrase is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model.

Logically, the query formulates a distance-threshold ϵ-join on the `actor_director_emb` vectors between the `imdb_t1` and `imdb_t2` tables. To comprehensively evaluate the engine across varying linkage densities, the parameter τ is strictly sampled from predefined discrete semantic similarity tiers $(0.40, 0.45, 0.50, 0.55, 0.60)$. These distinct values map directly to physical real-world relationships: a tight threshold ($\tau = 0.40$) captures highly identical cast combinations yielding an intermediate join selectivity of 0.01%, a medium

threshold ($\tau = 0.50$) captures similar associations yielding 0.23% selectivity, while a relaxed threshold ($\tau = 0.60$) captures loosely related connections yielding an exponentially larger 5.13% intermediate join selectivity. The semantic scoring signal ($\mathcal{S}$) is uniformly assigned to target either `plot_emb` or `title_emb` on either the `imdb_t1` or `imdb_t2` table. Crucially, to prevent a single highly-ranked anchor film from dominating the result set with multiple join partners, the valid joined pairs are explicitly deduplicated by the primary key (`tconst`) of the scored table. The pairs are then globally ranked by the L_2 distance between the assigned target embedding and the query vector, retaining the overall top $K = 20$ distinct pairs.

```
-- Q: Find classic-modern movie pairs with similar cast/
     crew (L2 <= 0.40),
-- ensure distinct classic movies, globally ranked by how
      well t1 fits a plot intent
SELECT t1_tconst, t1_title, t2_title
FROM (
  SELECT DISTINCT ON (t1.tconst)
         t1.tconst AS t1_tconst,
         t1.title AS t1_title,
         t2.title AS t2_title,
         t1.plot_emb <-> '[-0.007, -0.028, ...]' AS score
  FROM imdb_t1 t1
  JOIN imdb_t2 t2
    ON (t1.actor_director_emb <-> t2.actor_director_emb)
     <= 0.40
    -- cast similarity join (sampled from discrete
     similarity tiers)
  ORDER BY t1.tconst, score
) distinct_pairs
ORDER BY score
LIMIT 20;
```

Listing 6: Example W5 Query

W6: Pair up similar-cast films that fit a plot theme, within genre/era on each side (Filtered join). Building upon W5, workload W6 introduces exact structural predicates ($\mathcal{P}$) on both sides of the semantic join to evaluate filtered inter-entity synthesis. At a high level, this query pattern represents a constrained cross-era discovery task: it finds pairs of classic and modern movies that share a highly identical demographic cast or crew, satisfying rigorous structural constraints on both sides (restricting release eras or demanding specific minimum ratings), and ranks these linked pairs globally based on how well the primary movie matches a specific narrative search intent. To construct this workload, we synthesize 100 movie plot motifs using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] short phrases describing a specific movie theme, setting, or plot motif. Examples: 'a detective solving a murder in a small town', 'spaceship crew encountering an alien anomaly'. Output one phrase per line, no numbering, no other text."* Each generated phrase is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model to serve as the ranking intent on `plot_emb`.

To instantiate the structured predicates, each query is dynamically augmented with relational filters applied strictly to the metadata of both `imdb_t1` and `imdb_t2`. The predicate generation space is strictly bounded by explicit logical rules: temporal constraints applying the $\geq$ or $\leq$ operators on `year` sampled within the interval $[1980, 2020]$; quality thresholds applying the $\geq$ or $\leq$ operators on `rating` sampled within $[1.0, 10.0]$; and categorical constraints applying the IN operator over random subsets of 12 predefined cinematic `genres`. The join threshold tiers, deduplication policy, and sparse-to-dense linkage settings follow W5. W6 adds only dual-sided relational filtering before final ranking, and then ranks surviving pairs by the L_2 distance between `imdb_t1.plot_emb` and the query vector (top $K = 20$).

```
-- Q: Find thriller plot movies in imdb_t1 linked to
     imdb_t2 movies with
-- similar cast/crew (L2 <= 0.45), satisfying specific
     year and rating filters
SELECT t1_tconst, t1_title, t2_title
FROM (
  SELECT DISTINCT ON (t1.tconst)
         t1.tconst AS t1_tconst,
         t1.title AS t1_title,
         t2.title AS t2_title,
         t1.plot_emb <-> '[-0.007, -0.027, ...]' AS score
  FROM imdb_t1 t1
  JOIN imdb_t2 t2
    ON (t1.actor_director_emb <-> t2.actor_director_emb)
     <= 0.45
    -- cast similarity join (sampled from discrete
     similarity tiers)
  WHERE t1.year >= 1985
    AND t2.rating >= 5.7
  ORDER BY t1.tconst, score
) distinct_pairs
ORDER BY score
LIMIT 20;
```

Listing 7: Example W6 Query

W7: Pair up similar-cast films across two plot themes (Multi-scoring join). Building upon W5, workload W7 introduces complex multi-signal aggregation (Multi-$\mathcal{S}$) into the inter-entity synthesis pipeline ($\mathcal{J}$) without structural predicates ($\mathcal{P} = \emptyset$). At a high level, this query pattern represents a dual-intent cross-era discovery task: it finds tuples of classic and modern movies that share a highly identical demographic cast or crew, and globally ranks these pairs based on how well the primary movie matches a first narrative motif, how well the secondary movie matches a second narrative motif, and the absolute tightness of their shared cast similarity. To construct this workload, we synthesize 1000 search intents per table using OpenAI GPT-4o (temperature = 1.0) with the following exact prompt: *"Generate exactly [N] short, diverse phrases (3-8 words each) that someone might type when searching for movies. They should cover a wide range of genres, moods, settings, and themes. Avoid repeating themes already covered.\nOutput one phrase per line, no numbering, no other text."* Each phrase is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model to target either the `imdb_t1 plot_emb` or `imdb_t2 plot_emb`. The primary and secondary query vectors are paired dynamically across specific correlation tiers (anti, low, medium, high) to rigorously control query hardness.

The join threshold tiers and sparse-to-dense linkage settings follow W5. The key change in W7 is the ranking objective: globally valid pairs are scored by a late-binding weighted sum of three L_2 distances, combining `imdb_t1.plot_emb` to the first intent, `imdb_t2.plot_emb` to the second intent, and the cross-table `actor_`

director_emb distance. Each weight vector is sampled and normalized to 1.0, and the top $K = 20$ pairs are retained.

```
-- Q: Find classic-modern movie tuples with similar cast/
     crew (L2 <= 0.50),
-- ranked globally by a 3-way multi-scoring objective
SELECT t1.tconst AS t1_tconst,
       t2.tconst AS t2_tconst,
       ( 0.20 * (t1.plot_emb <-> '[-0.009, 0.021, ...]')
         -- primary plot intent (L2 distance)
       + 0.20 * (t2.plot_emb <-> '[-0.019, -0.008, ...]')
         -- secondary plot intent (L2 distance)
       + 0.60 * (t1.actor_director_emb <-> t2.
     actor_director_emb)
         -- actual cast similarity (L2 distance)
       ) AS score
FROM imdb_t1 t1
JOIN imdb_t2 t2
  ON (t1.actor_director_emb <-> t2.actor_director_emb) <=
      0.50
  -- cast similarity join (sampled from discrete
     similarity tiers)
ORDER BY score
LIMIT 20;
```

Listing 8: Example W7 Query

W8: Pair up similar-cast films across two plot themes, within genre/era on each side (Filtered multi-scoring join). Building upon W7, workload W8 represents the most comprehensive query pattern, introducing exact structural predicates ($\mathcal{P}$) on both sides of the multi-signal aggregation (Multi-$\mathcal{S}$) and inter-entity synthesis pipeline ($\mathcal{J}$). At a high level, this query pattern represents a constrained, dual-intent cross-era discovery task: it finds tuples of classic and modern movies that share a highly identical demographic cast or crew, satisfying rigorous structural constraints on both sides (restricting release eras or demanding specific minimum ratings), and globally ranks these linked pairs based on how well the primary movie matches a first narrative motif, how well the secondary movie matches a second narrative motif, and the absolute tightness of their shared cast similarity. To construct this workload, we synthesize 100 pairs of distinct movie concepts using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] pairs of movie concepts. Each line should contain two distinct concepts separated by a pipe character '|'. For example: 'A gritty sci-fi thriller about time travel | A lighthearted comedy about a robot learning to love'. Output only the pipe-separated pairs, one pair per line."* Each text field in the generated pair is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model, with the first concept targeting the imdb_t1 plot_emb and the second targeting the imdb_t2 plot_emb.

To instantiate the structured predicates, each query is dynamically augmented with relational filters applied strictly to the metadata of both imdb_t1 and imdb_t2. The predicate generation space is strictly bounded by explicit logical rules: temporal constraints applying the $\geq$ or $\leq$ operators on year sampled within the interval $[1980, 2020]$; quality thresholds applying the $\geq$ or $\leq$ operators on rating sampled within $[1.0, 10.0]$; and categorical constraints applying the IN operator over random subsets of 12 predefined cinematic genres. W8 follows W5 for join tiers, and follows W7 for three-signal late-binding ranking; its only additional step is applying dual-sided relational filters before ranking. The final result keeps the top $K = 20$ distinct pairs.

```
-- Q: Find "Sci-fi movies with robots" in imdb_t1 (year
     >= 1985) linked to
-- "Exploration documentaries of outer space" in imdb_t2
     (rating >= 5.7)
-- with similar cast/crew (L2 <= 0.50), ranked globally
     by a 3-way objective
SELECT t1.tconst AS t1_tconst,
       t1.title AS t1_title,
       t2.title AS t2_title,
       ( 0.25 * (t1.plot_emb <-> '[-0.009, 0.021, ...]')
         -- primary plot intent (L2 distance)
       + 0.25 * (t2.plot_emb <-> '[-0.027, -0.013, ...]')
         -- secondary plot intent (L2 distance)
       + 0.50 * (t1.actor_director_emb <-> t2.
     actor_director_emb)
         -- actual cast similarity (L2 distance)
       ) AS score
FROM imdb_t1 t1
JOIN imdb_t2 t2
  ON (t1.actor_director_emb <-> t2.actor_director_emb) <=
      0.50
  -- cast similarity join (sampled from discrete
     similarity tiers)
WHERE t1.year >= 1985
  AND t2.rating >= 5.7
ORDER BY score
LIMIT 20;
```

Listing 9: Example W8 Query

A.2 MOLECULE Benchmark

A.2.1 Dataset Description. Unlike IMDB, which is positioned as a fabricated discovery benchmark, MOLECULE provides real hybrid workloads extracted from an operational Biofoundry project dedicated to academic knowledge integration and retrieval. The underlying database naturally stores heterogeneous entities—including research papers, researchers, organizations, and biochemical concepts—augmented with semantic tags, structured attributes, and multiple field-level embeddings. To expose this complex, evidence-centric biomedical reasoning in a relational benchmark, we project the relevant entity interactions into a bipartite schema: a fact-side relation (molecule_fact) and a paper-side relation (molecule_paper). This layout successfully captures the target user behavior of our deep research agent system: proposing molecule-related concepts, retrieving supporting literature, applying metadata constraints, and ranking candidates under multi-signal criteria.

A.2.2 Data Source and Construction. To construct this benchmark, we combine two sources: (i) molecule-centric fact/paper data extracted from the broader heterogeneous knowledge graph and (ii) deep research agent query traces. The construction pipeline is:

- **Agent-Trace Grounding:** We collect historical deep research agent queries and cluster them into eight workload intents (W1–W8). For each intent class x, we retain a set of representative trace templates T_x, where each template includes the query skeleton and the fields that must be instantiated (query_text, predicates, join tier, weight vector, or combinations thereof).

- **Data Sourcing:** We collect molecule facts and publication records with complete text fields and normalized molecule identifiers from the Biofoundry integration engine, then retain only rows with valid metadata required by the template instantiation fields.
- **Profile Construction:** On the fact side, each row stores a fact statement plus structured molecule descriptors; on the paper side, each row stores publication metadata and abstract-level content. This yields a physically separated but semantically linkable two-relation layout aligned with agent reasoning hops.
- **Vectorization / Structural Features:** We encode textual fields with Gemini-embedding-001 into 1536-dimensional vectors and represent molecular structure with 1024-bit ECFP fingerprints. These representations are exactly the fields consumed by template-driven augmentation from T_x.

A.2.3 Workload Construction. To systematically evaluate execution strategies, we construct eight workloads that cover the full taxonomy of our query model: structured predicates ($\mathcal{P}$), multi-signal scoring ($\mathcal{S}$), and semantic joins ($\mathcal{J}$). For each workload x, we follow an agent-grounded augmentation recipe: (1) select representative templates T_x from deep research agent traces, (2) map template slots to physical fields, (3) generate or sample slot values, and (4) materialize augmented queries as workload JSON.

Predicates and Scoring ($\mathcal{P}$ and $\mathcal{S}$). For fact-side filtering, predicate atoms are sampled from categorical fields (`fact_type`, `claim_polarity`, `mol_source_origin`, `mol_toxicity_flag`, `mol_moa`, `mol_target_type`, `evidence_type`, `organism`), numeric fields (`mol_mw`, `mol_logp`, `mol_tpsa`, `mol_num_atoms`, `mol_num_rings`, `mol_num_hbd`, `mol_num_hba`), and array containment on `mol_therapeutic_area`. Multi-signal scoring combines fact-text semantic distance and molecule-structure Jaccard distance, with per-query randomized weights normalized to sum to 1.

Semantic Joins ($\mathcal{J}$). For inter-entity synthesis (W5–W8), we link fact rows to paper rows using a distance-threshold join over textual evidence vectors. The threshold τ is sampled from discrete tiers $\{0.45, 0.50, 0.55, 0.65\}$, labeled *very_similar*, *similar*, *related*, and *loose*. These tiers induce exponentially different candidate densities (from thousands to millions of pairs), which we intentionally preserve to stress both sparse and dense join regimes.

To avoid synthetic key joins, all cross-table linkage is parameterized geometrically by τ; smaller tiers produce strict semantic evidence alignment, and larger tiers expand recall with sharply higher pair counts. This discrete-tier design mirrors realistic retrieval behavior where semantic relatedness appears as tail events in embedding space.

W1: Find facts on a research topic (Semantic retrieval). For W1, we first extract a template set T_1 from agent traces whose intent is single-hop fact retrieval without filters or joins. We then instantiate the `query_text` slot using OpenAI GPT-4o (temperature = 0.9) with the exact prompt: *"Generate exactly [N] diverse natural-language queries a biomedical researcher might type when searching a database of facts about drug molecules and proteins. Cover varied topics: drug mechanism of action, toxicity, pharmacokinetics, structure-activity relationships, clinical outcomes, natural products, therapeutic areas (oncology / neurology / infectious / metabolic / cardio / psychiatry / immunology), organisms tested (human / mouse / rat / cell_line / bacteria), in vitro vs in vivo evidence, and synthesis routes. Vary phrasing style: keyword style, question style, long-sentence style. Keep each under 15 words, realistic, no numbering, one phrase per line."* Instantiated queries are deduplicated, embedded by Gemini-embedding-001 (1536-d), and bound to `molecule_fact.fact_emb` with $K = 20$. This yields pure semantic retrieval with $\mathcal{P} = \emptyset$ and $\mathcal{J} = \emptyset$.

```
-- Q: "inhibits beta-amyloid aggregation"
SELECT id, organism, property_type, fact_description
FROM molecule_fact
ORDER BY fact_emb <-> '[-0.0125, 0.0341, ...]'
LIMIT 20;
```

Listing 10: Example W1 Query

W2: Find facts on a research topic, within organism/property scope (Filtered retrieval). For W2, we extract templates T_2 from agent traces where semantic retrieval is explicitly conditioned by metadata constraints. We instantiate `query_text` by referring to the W1 generation setting, then augment each template with sampled predicate slots (1–3 predicates) over `molecule_fact`. The predicate slot space is explicitly bounded: categorical IN predicates over `fact_type`, `claim_polarity`, `mol_source_origin`, `mol_toxicity_flag`, `mol_moa`, `mol_target_type`, `evidence_type`, `organism`; numeric threshold predicates ($\geq, \leq$) over `mol_mw`, `mol_logp`, `mol_tpsa`, `mol_num_atoms`, `mol_num_rings`, `mol_num_hbd`, `mol_num_hba`; and array containment (`@>`) on `mol_therapeutic_area`. We retain augmented queries by selectivity buckets: *tight* $[0.01, 0.10)$ at 30%, *medium* $[0.10, 0.30)$ at 40%, and *loose* $[0.30, 0.70)$ at 30%.

```
-- Q: "drug interaction mechanisms with cytochrome P450
    enzymes"
-- medium-selectivity bundle: numeric + categorical +
    array containment
SELECT id, organism, property_type, fact_description
FROM molecule_fact
WHERE mol_tpsa >= 115.54
  AND organism IN ('human', 'mouse')
  AND mol_therapeutic_area @> ARRAY['oncology']
ORDER BY fact_emb <-> '[-0.0102, -0.0001, ...]'
LIMIT 20;
```

Listing 11: Example W2 Query

W3: Find facts on a topic about a reference molecule (Multi-scoring retrieval). For W3, we extract templates T_3 from traces where the agent jointly reasons over textual evidence and molecule structure in one retrieval hop. We instantiate two template slots: (i) `query_text` (referring to the W1 generation setting) mapped to `molecule_fact.fact_emb` and (ii) a reference molecule slot mapped to `molecule_fact.mol_ecfp` as a 1024-bit ECFP anchor. We then instantiate per-query weight slots and normalize to 1.0, producing a late-binding weighted sum across L2 text distance and Jaccard structure distance.

```
-- Q: "drug interaction mechanisms..." related to a
    specific reference molecule
SELECT id, organism, property_type, fact_description
FROM molecule_fact
ORDER BY
  0.60 * (fact_emb <-> '[-0.0101, -0.0001, ...]')
  -- biological topic intent (semantic distance)
```

```
+ 0.40 * (mol_ecfp <%> '[0, 0, 0, 1, ...]')
  -- structural chemical similarity (Jaccard distance)
LIMIT 20;
```

Listing 12: Example W3 Query

W4: Find facts on a topic about a reference molecule, within organism/property scope (Filtered multi-scoring). For W4, we extract templates T_4 from traces that combine dual-signal scoring with explicit fact-side filtering. We instantiate text, structure, and weight slots exactly as in W3, then instantiate 1–3 predicate slots by referring to the W2 predicate setting and retain queries under the same selectivity-bucket policy (*tight* 30%, *medium* 40%, *loose* 30%). This yields a template-grounded filtered multi-scoring workload on a single relation.

```
-- Q: "drug interaction mechanisms..." related to a
     specific reference molecule
-- filtered multi-scoring: same two signals as W3 +
     predicates following W2
SELECT id, organism, property_type, fact_description
FROM molecule_fact
WHERE mol_tpsa >= 115.54
  AND evidence_type IN ('in_vitro', 'in_vivo')
  AND mol_target_type IN ('enzyme', 'receptor')
ORDER BY
  0.60 * (fact_emb <-> '[-0.0101, -0.0001, ...]')
  -- biological topic intent (semantic distance)
+ 0.40 * (mol_ecfp <%> '[0, 0, 0, 1, ...]')
  -- structural chemical similarity (Jaccard distance)
LIMIT 20;
```

Listing 13: Example W4 Query

W5: Find paper-supported facts about a reference molecule (Semantic join). For W5, we extract templates T_5 from traces where the agent retrieves fact candidates and then links them to supporting literature. We instantiate a structure-anchor slot (sampled valid `mol_id`, 1024-bit ECFP) and a join-threshold slot for `molecule_fact.fact_emb` ↔ `molecule_paper.abstract_emb`. The join-threshold slot is sampled from discrete tiers: `very_similar` (0.45), `similar` (0.50), `related` (0.55), `loose` (0.65), and `unrelated` (0.75), preserving the observed sparse-to-dense evidence-linking regimes in agent behavior.

```
-- Q: Find facts matching a specific molecular structure,
-- then join papers providing semantic textual evidence (
     L2 <= 0.45)
SELECT fact_id, paper_id
FROM (
  SELECT DISTINCT ON (f.id)
         f.id AS fact_id,
         p.pmid AS paper_id,
         (f.fact_emb <-> p.abstract_emb) AS join_dist
  FROM (
    SELECT id, fact_emb
    FROM molecule_fact
    ORDER BY mol_ecfp <%> '[0, 0, 0, 0, ...]'  -- fact-
     side structure intent
    LIMIT 20
  ) AS f
  JOIN molecule_paper AS p
    ON (f.fact_emb <-> p.abstract_emb) <= 0.45   --
     very_similar tier
  ORDER BY f.id, join_dist
) AS dedup
ORDER BY join_dist
LIMIT 20;
```

Listing 14: Example W5 Query

W6: Find paper-supported facts about a reference molecule, within fact/paper filters (Filtered join). For W6, we extract templates T_6 from traces where evidence-linking is constrained by both fact-side and paper-side metadata. We instantiate W5 structure-anchor and join slots, then instantiate dual-sided predicate slots using an oversampled candidate pool: fact-side predicates by referring to the W2 predicate setting and paper-side predicates over `year`, `journal`, `pub_type`, and `mesh_terms`. Candidate templates are retained via soft selectivity acceptance (knee at 0.10), producing a broad left/right selectivity grid aligned with agent-side constrained retrieval behavior.

```
-- Q: Facts for a reference molecule (natural/microbial
     origin),
-- joined with US Gov supported papers providing semantic
     evidence
SELECT fact_id, paper_id
FROM (
  SELECT DISTINCT ON (f.id)
         f.id AS fact_id,
         p.pmid AS paper_id,
         (f.fact_emb <-> p.abstract_emb) AS join_dist
  FROM (
    SELECT id, fact_emb
    FROM molecule_fact
    WHERE mol_source_origin IN ('natural_product', '
     microbial')
      AND mol_toxicity_flag IN ('non_toxic')
    ORDER BY mol_ecfp <%> '[0, 0, 0, 0, ...]'
    LIMIT 20
  ) AS f
  JOIN molecule_paper AS p
    ON (f.fact_emb <-> p.abstract_emb) <= 0.45
   AND p.pub_type @> ARRAY['Research Support, U.S. Gov''t
     ']
   AND p.mesh_terms @> ARRAY['Humans']
  ORDER BY f.id, join_dist
) AS dedup
ORDER BY join_dist
LIMIT 20;
```

Listing 15: Example W6 Query

W7: Find paper-supported facts combining a reference molecule, a topic, and tight fact-paper alignment (Multi-scoring join). For W7, we extract templates T_7 from traces where the agent performs cross-table tuple retrieval with three simultaneous ranking signals. We instantiate a molecule-anchor slot, a topic-text slot, a join-tier slot, and a three-weight slot. Topic texts are generated by OpenAI GPT-4o (temperature = 0.8) from four exact prompt templates (`mol_specific`, `area_specific`, `target_topic`, `unrelated`) and embedded by Gemini-embedding-001 onto `molecule_paper.abstract_emb`. Join tiers are sampled from $\{0.45, 0.50, 0.55, 0.65\}$ and weight slots are normalized to 1.0, yielding a template-grounded late-binding multi-signal join workload for global tuple pairs.

```
-- Q: Find fact-paper tuples matching a molecular
     structure and topic,
-- ranked globally by a 3-way objective
SELECT f.id AS fact_id,
       p.pmid AS paper_id,
```

```
       ( 0.35 * (f.mol_ecfp <%> '[0, 1, 0, ...]')      -- fact molecule intent
       + 0.33 * (p.abstract_emb <-> '[-0.009, 0.021, ...]') -- paper topic intent
       + 0.32 * (f.fact_emb <-> p.abstract_emb) ) AS score  -- join penalty
FROM molecule_fact AS f
JOIN molecule_paper AS p
  ON (f.fact_emb <-> p.abstract_emb) <= 0.50  -- similar tier
ORDER BY score
LIMIT 20;
```

Listing 16: Example W7 Query

W8: Find paper-supported facts combining molecule/topic/fact-paper alignment, within fact/paper filters (Filtered multi-scoring join). For W8, we extract templates T_8 from traces representing full deep-research behavior: constrained cross-entity tuple linking with multi-signal ranking. We instantiate W7 slots (molecule anchor, topic embedding, join tier, normalized three-weight vector) and W6 slots (dual-sided predicate bundle) together. Operationally, we pair an oversampled W6 predicate pool with W7 query templates, then select a final subset that preserves diversity over filter hardness, join density tier, and topic-to-molecule correlation tier. This yields the comprehensive agent-grounded tuple workload where $\mathcal{P}$, $\mathcal{S}$, and $\mathcal{J}$ are jointly active.

```
-- Q: Find tuples of Catalyst facts + NIH-funded papers, ranked
-- globally by molecule intent + topic intent + alignment penalty
SELECT f.id AS fact_id,
       p.pmid AS paper_id,
       ( 0.35 * (f.mol_ecfp <%> '[0, 1, 0, ...]')
       + 0.33 * (p.abstract_emb <-> '[-0.006, 0.012, ...]')
       + 0.32 * (f.fact_emb <-> p.abstract_emb) ) AS score
FROM molecule_fact AS f
JOIN molecule_paper AS p
  ON (f.fact_emb <-> p.abstract_emb) <= 0.55   -- related tier
WHERE f.mol_moa IN ('catalyst')
  AND f.evidence_type IN ('in_vivo', 'clinical')
  AND p.pub_type @> ARRAY['Research Support, N.I.H., Extramural']
  AND p.year >= 2005
ORDER BY score
LIMIT 20;
```

Listing 17: Example W8 Query

Appendix B Integration with Semantic Operator Systems

As established in Sec. 2.1, DASE is designed to assist *semantic operator systems* [37, 43, 47] by acting as a cost-effective prefiltering stage that prunes input data before any LLM invocation. These systems typically implement a suite of LLM-backed *semantic operators*, including SEM_FILTER, SEM_CLASSIFY, SEM_RANK, SEM_JOIN, and SEM_MAP. DASE functions as a plug-in prefilter for these operators, leveraging multi-facet vector embeddings to identify the most promising candidate tuples for downstream LLM evaluation. Detailed SemBench performance results are provided in Table 3.

B.1 The Prefilter Pipeline

In this architecture, DASE serves as a pluggable prefilter component. The overall workflow typically operates in two stages: DASE first recommends a high-recall candidate set, which a full-featured semantic operator engine subsequently evaluates and refines using targeted LLM calls. Alternatively, in scenarios where the downstream semantic operator is omitted, the workflow bypasses LLM evaluation entirely and simply accepts all of DASE's multi-dimensional recommendations directly as the final output. This "DASE alone" configuration (denoted as *DASE* in Table 3) remains highly competitive whenever the embedding distance effectively separates relevant tuples from the background distribution.

B.2 Per-Operator Strategies

The integration logic is specifically tailored to the semantics of each operator. Crucially, DASE treats every natural-language query as a *multi-dimensional evaluation task*: it decomposes the user's semantic intent into a statistical proxy comprising weighted aggregations over multiple, potentially heterogeneous embedding streams (e.g., visual, textual, and structural). At the execution layer, these operators are natively compiled into the multi-scoring architecture introduced in Sec. 4, where independent Filtered-Score Streamers (FSS) are synchronized and pruned by the Multi-Scoring Aggregator (MSA).

B.2.1 SEM_FILTER. A SEM_FILTER evaluates a natural-language Boolean predicate ℓ on an input relation T. For a query such as "finding a blue shirt with positive reviews," the semantics are not restricted to a single modality. DASE captures this by modeling the filter as a joint multi-scoring task across all relevant feature spaces $\mathcal{F}$ (e.g., image embeddings for visual traits and text embeddings for sentiment).

For each space $f \in \mathcal{F}$, DASE derives a specific set of positive anchors P_f and antithetical anchors N_f. The MSA then synchronizes these independent FSS streams to compute a cross-dimensional contrastive margin on the fly:

$$s_i = \sum_{f \in \mathcal{F}} w_f \left(\frac{1}{|P_f|} \sum_{p \in P_f} \cos(\mathbf{e}_p, \mathbf{e}_i^{(f)}) - \frac{1}{|N_f|} \sum_{n \in N_f} \cos(\mathbf{e}_n, \mathbf{e}_i^{(f)}) \right)$$

where $\mathbf{e}_i^{(f)}$ is the embedding of tuple i in space f, and w_f balances the contribution of each modality. This mechanism natively aggregates signals—such as visual blueness and textual sentiment—to identify top candidates. Because DASE acts as a high-recall prefilter, avoiding false negatives is paramount. We employ two recommendation rules that utilize intentionally relaxed boundaries:

- *Margin-based uncertainty band.* For total labeling tasks (e.g., *Wildlife* Q1), DASE applies thresholds to the joint margin s_i. To aggressively protect recall, it forwards a conservatively wide uncertain band (typically $\alpha \approx 0.2$–0.5) for LLM evaluation, while accepting the cheap sign of s_i for confidently extreme rows.
- *Top-K' shortlist.* For `LIMIT` K queries (e.g., *Movie* Q1), the MSA seamlessly streams the K' tuples with the highest aggregate s_i. The downstream engine's `AI.IF` then evaluates this multi-dimensional shortlist, short-circuiting at the K-th

valid survivor. This heuristic reduces the query latency for *Movie* Q1 from 26.3 s to 0.5 s.

B.2.2 SEM_CLASSIFY. A SEM_CLASSIFY operator assigns each tuple to one of K enumerated classes $C = \{c_1, \ldots, c_K\}$. Whereas SEM_FILTER contrasts a single positive concept against its antithetical paraphrases, SEM_CLASSIFY generalises this to a K-way contrast: each class becomes its own anchor, and the per-row decision reduces to identifying which anchor wins. For instance, in *Cars* Q10 the engine must classify automotive complaints into 24 distinct problem categories, where a complaint may comprise both a textual description and an image of the defect, requiring multi-modal reasoning.

DASE natively maps this multi-class evaluation onto a parallel multi-scoring retrieval task. It embeds each class anchor prompt c_k once per facet to obtain $\mathbf{e}_{c_k}^{(f)}$, initialising an independent FSS stream per category. The MSA then synchronises these K streams across the required feature spaces $\mathcal{F}$ (e.g., visual embeddings of the damaged car part and text embeddings of the complaint description), aggregating per-facet cosine similarities into a per-class affinity $s_i^{(k)}$, from which a proxy label $\hat{k}_i$ and confidence margin m_i follow directly:

$$s_i^{(k)} = \sum_{f \in \mathcal{F}} w_f \cos\!\left(\mathbf{e}_{c_k}^{(f)}, \mathbf{e}_i^{(f)}\right), \; k = 1, \ldots, K,$$

$$\hat{k}_i = \arg\max_k s_i^{(k)}, \; m_i = s_i^{(\hat{k}_i)} - \max_{k \neq \hat{k}_i} s_i^{(k)}.$$

All three quantities are computed without any LLM invocation. DASE then partitions the result set on m_i: the bottom-α tuples (smallest margin) are forwarded to the downstream engine for rigorous refinement via `AI.CLASSIFY`, while the engine safely accepts the cheap argmax $\hat{k}_i$ on the remaining confident rows. The final output unions the LLM-resolved tail with the DASE-asserted confident bulk, achieving macro F_1 parity with an exhaustive all-LLM baseline at roughly half the inference cost (Table 3).

B.2.3 SEM_RANK. A SEM_RANK operator orders or scores tuples according to a defined rubric. DASE compiles this requirement into two distinct multi-scoring execution paths based on the query objective:

- *Top-K retrieval.* When the query strictly demands the highest-ranking candidates, the execution naturally reduces to the SEM_FILTER + `LIMIT` paradigm of Sec. B.2.1. DASE leverages the MSA to synchronise contrastive margin streams across all targeted feature spaces $\mathcal{F}$, organically short-circuiting the retrieval once K confidently top-ranked tuples survive downstream `AI.IF` validation.
- *Rubric scoring.* When a strict numeric label is required for *every* row (e.g., *Movie* Q9, which scores from 1 to 5 how much a reviewer liked a movie), the task is structurally a SEM_CLASSIFY over an *ordinal* class set: each rubric grade $g \in \{1, \ldots, G\}$ plays the role of a class anchor c_g, and the per-grade affinity $s_i^{(g)}$, proxy label $\hat{g}_i = \arg\max_g s_i^{(g)}$, and confidence margin m_i all carry over verbatim from the formulation in Sec. B.2.2. The only operator-specific change lies in the cascade policy. Because global rank-correlation metrics like Spearman integrate over the entire score distribution, a single mis-ranked tuple can disproportionately degrade the global metric, so DASE cannot afford the aggressive accept-rate used for nominal classification. The bottom-α tail (smallest m_i) is therefore widened substantially ($\alpha \approx 0.7$) and refined via `AI.SCORE` rather than `AI.CLASSIFY`; the proxy argmax $\hat{g}_i$ is accepted only on the confident head, and the two streams are concatenated into the final ranking.

B.2.4 SEM_JOIN. A SEM_JOIN evaluates a match predicate over pairs $(t_i, t_j) \in T_1 \times T_2$, such as verifying if two items share the same brand and category (*Ecomm* Q7). Unlike the per-row operators above, the candidate space here is the Cartesian product $|T_1| \cdot |T_2|$, so prefiltering is not merely an optimisation but a feasibility requirement. DASE evaluates the match proxy directly against the SemJI, which strictly decouples continuous multi-facet similarities in `SemJI(embed)` from discrete semantic truths in `SemJI(AI.IF)`.

During execution, DASE first probes `SemJI(AI.IF)` to resolve previously verified pairs at zero LLM cost. For unresolved pairs, the MSA translates the NL match criteria into a multi-scoring problem, synchronising the traversal across multiple `SemJI(embed)` streams (e.g., product titles *and* images) to compute a joint similarity score.

- *Two-sided uncertainty band.* For joins like *Ecomm* Q7, the MSA organically identifies pairs with the highest aggregate scores ($S_{ij} \geq \tau_{\text{hi}}$) as confident matches. Pairs in the intermediate range ($\tau_{\text{lo}} < S_{ij} < \tau_{\text{hi}}$)—where combined visual and textual cues are strong but exact identity is ambiguous—are forwarded to the LLM. The MSA traversal short-circuits once the upper bound of the joint score drops below τ_{lo}, implicitly pruning the massive tail of certain negatives. If the materialised caches exhaust before reaching τ_{lo}, DASE gracefully resumes the stream by dynamically evaluating distances via the base vector indices as discussed in Sec. 3.2.
- *Top-K poles.* For queries seeking extreme pairs (e.g., *Movie* Q5), the MSA fetches the K highest- and lowest-scoring pairs by directly aggregating the underlying facet streams.

Crucially, as the downstream LLM resolves uncertain cases via `AI.IF`, the deterministic results are written back exclusively to `SemJI(AI.IF)`, effectively amortising the cost of multi-faceted reasoning for subsequent queries.

B.2.5 SEM_MAP. A SEM_MAP call applies a per-row natural-language transformation that returns a free-form string, e.g., brand extraction (*Ecomm* Q3), colour labelling (*Ecomm* Q12), or genre clustering (*MMQA* Q4). Because the output domain is open-vocabulary, neither the contrastive margin of SEM_FILTER nor the K-way argmax of SEM_CLASSIFY applies—there is no fixed anchor set to contrast against. Instead, DASE exploits a complementary form of semantic redundancy: tuples whose embeddings are near-duplicates across the query-relevant feature spaces $\mathcal{F}$ are statistically guaranteed to map to the same target string, so a single LLM call on one representative safely labels its entire neighbourhood.

To compose this joint similarity, the MSA initialises a parallel FSS per facet $f \in \mathcal{F}$ and aggregates the per-facet cosine streams into a unified affinity $\sigma_{ij} = \sum_{f \in \mathcal{F}} w_f \cos(\mathbf{e}_i^{(f)}, \mathbf{e}_j^{(f)})$ on the fly. DASE

then clusters the input tuples by agglomerative clustering on this aggregate affinity (complete linkage, distance threshold τ_d=0.10, i.e., $\sigma_{ij} \geq 0.90$) and recommends only the medoid nearest each cluster centroid. The downstream engine runs `AI.GENERATE` on this condensed K-tuple shortlist (e.g., ∼90 medoids out of 500 rows on *Ecomm* Q3), and the returned label is broadcast to the entire cluster via a SQL join on the synthetic cluster ID. Because clusters form in the joint multi-facet space rather than over any single embedding axis, the medoid is robust to single-modality drift—visually similar but textually distinct items remain in separate clusters. Quality degrades only when τ_d is lax enough to fuse genuinely distinct entities, which on *Ecomm* costs less than 0.01 ARI while reducing LLM invocations by an order of magnitude.